



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

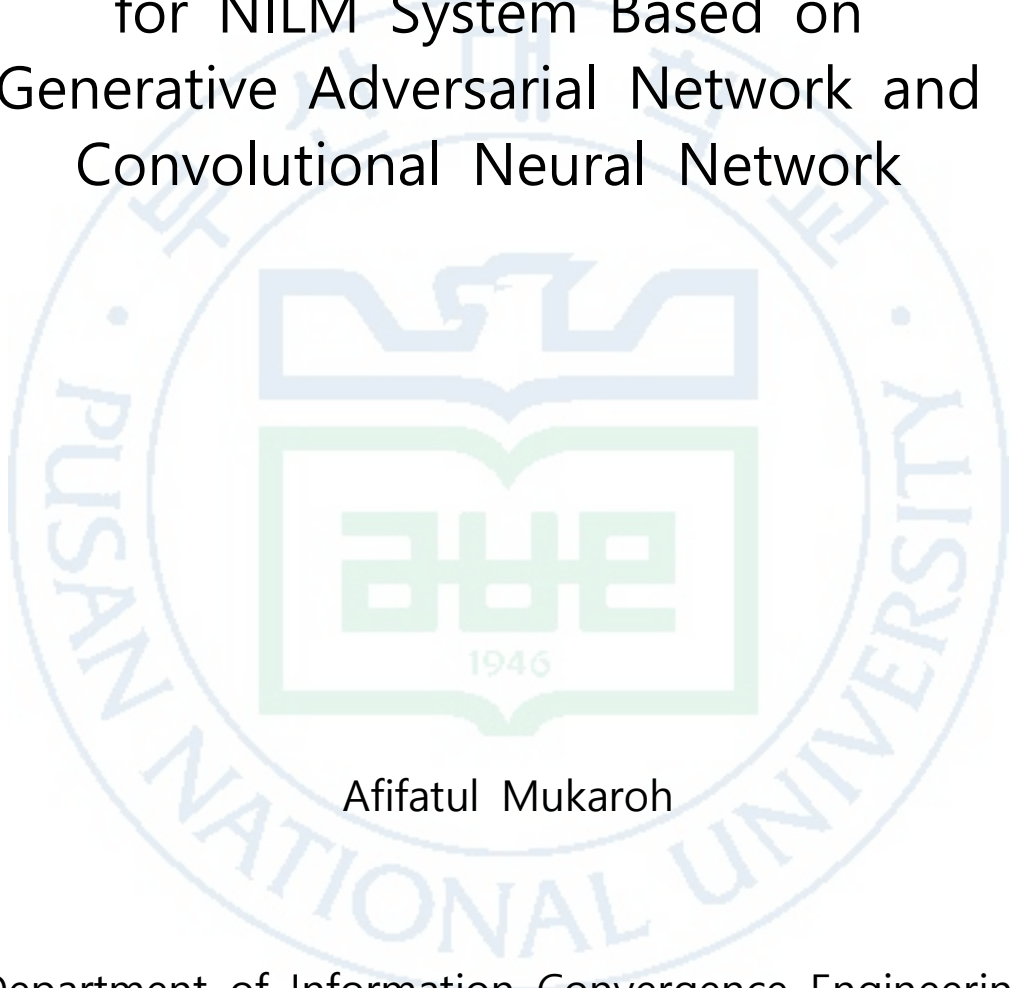
저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

Thesis for the degree of Master of Science

Two-Step Load Identification Method for NILM System Based on Generative Adversarial Network and Convolutional Neural Network

The background of the page features a large, light blue watermark of the Pusan National University seal. The seal is circular and contains a stylized green emblem in the center. The emblem consists of a crown-like shape above an open book, with the year '1946' inscribed below the book. The text 'PUSAN NATIONAL UNIVERSITY' is written in a circular path around the emblem.

Afifatul Mukaroh

Department of Information Convergence Engineering
The Graduate School
Pusan National University

February 2021

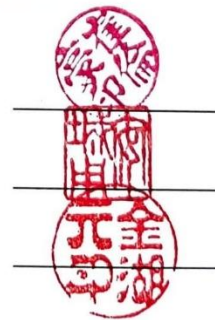
Two-Step Load Identification Method for NILM System Based on Generative Adversarial Network and Convolutional Neural Network

A Thesis submitted to the graduate school of
Pusan National University in partial fulfillment of
the requirements for the degree of Master of Science
under the direction of Howon Kim

The thesis for the degree of Master of Science
by Afifatul Mukaroh
has been approved by the committee members.

December 11, 2020

Chair	Yoon-Ho Choi
Member	Sungyong Ahn
Member	Howon Kim



Contents

I.	Introduction	1
A.	Research Motivation	1
B.	Related Work	2
C.	Thesis Contribution	4
D.	Thesis Outline	4
II.	Background Theory	5
A.	NILM	5
1.	NILM Objective	7
2.	Features in NILM	8
B.	GAN	12
1.	Conditional GAN	13
2.	Pix2pix GAN	13
C.	CNN	14
1.	Feature Extraction Part	15
2.	Classification Part	17
III.	Proposed Method	18
A.	General Framework	18
B.	Preprocessing	19
1.	Data Augmentation	21
2.	Spectrogram Transformation	22
C.	Data Splitting	23
D.	Model Generation	23

E.	Inference.....	24
IV.	Implementation and Experimental Result	30
A.	Environment Specification	30
B.	Dataset.....	30
1.	LIT-dataset.....	30
2.	Private Dataset	32
C.	Data Preprocessing and Data Splitting Result	33
D.	GAN Model Generation.....	38
E.	CNN Model Generation.....	41
F.	Inference.....	46
1.	Step 1-Denoising	46
2.	Step 2-Load Identification.....	48
V.	Conclusion.....	50

List of Figures

[Figure 1] Illustration of ILM and NILM difference	5
[Figure 2] Illustration of Aggregated Power [13]	6
[Figure 3] Illustration of Aggregated Power [14]	6
[Figure 4] Difference between low frequency and high frequency.....	9
[Figure 5] Illustration of steady state	10
[Figure 6] Illustration of transient state	11
[Figure 7] Illustration of Background Load and Mixed Load [10]	11
[Figure 8] GAN Training Illustration	12
[Figure 9] Conditional GAN Training illustration	13
[Figure 10] Illustration of CNN framework	15
[Figure 11] Convolution Illustration	16
[Figure 12] Max Pooling and Average Pooling Illustration.....	17
[Figure 13] Fully Connected Layer Illustration	17
[Figure 14] Main Process of Proposed Method.....	19
[Figure 15] Preprocessing	20
[Figure 16] PSD of White Noise	21
[Figure 17] Gaussian Probability Distribution.....	22
[Figure 18] Illustration Of 10-Fold Validation in Proposed Method.....	23
[Figure 19] Illustration of Model Generation.....	24
[Figure 20] Illustration of Inference Process	25
[Figure 21] Pix2pix GAN Architecture [10].....	26
[Figure 22] Encoder Block Components [10].....	26
[Figure 23] Decoder Block Components [10].....	27
[Figure 24] Discriminator Architecture of Pix2pix GAN. [10].....	27
[Figure 25] Convolutional Neural Network (CNN) architecture [10]	28
[Figure 26] Augmentation Results Comparison	35
[Figure 27] Spectrogram Transformation of Mixed Load [10].....	36
[Figure 28] Spectrogram Transformation of Single Load [10]	36

[Figure 29] K-Fold Cross Validation Implemented in LIT-Dataset	37
[Figure 30] K-Fold Cross Validation Implemented in Private Dataset	37
[Figure 31] Generator Loss Curve from GAN Training Result - LIT Dataset [10].....	39
[Figure 32] Discriminator Loss from GAN Training Result - LIT dataset [10].....	40
[Figure 33] Generator Loss from GAN Training Result - Private Dataset.....	40
[Figure 34] Discriminator Loss GAN Training Result - Private Dataset	41
[Figure 35] CNN Training Result- LIT Dataset.....	43



List of Tables

[Table 1] Previous Research.....	2
[Table 2] Mixed Load Dataset from LIT-Dataset	31
[Table 3] Single Load Dataset from LIT-Dataset.....	31
[Table 4] Mixed Load Dataset from Private Dataset	32
[Table 5] Single Load Dataset from Private Dataset.....	33
[Table 6] Mixed Load Dataset Splitting - LIT Dataset	38
[Table 7] Mixed Load Dataset Splitting - Private Dataset	38
[Table 8] GAN Model Hyperparameter	38
[Table 9] CNN Model Hyperparameter	41
[Table 10] CNN Cross Validation - LIT Dataset	42
[Table 11] CNN Testing Result - LIT Dataset	44
[Table 12] Comparison Between Proposed CNN Architecture and VGG16.....	45
[Table 13] CNN Cross Validation - Private Dataset.....	45
[Table 14] CNN Testing Result - Private Dataset	46
[Table 15] GAN Inference Result of LED Lamp - LIT Dataset	47
[Table 16] GAN Inference Result of Hairdryer - Private Dataset	47
[Table 17] CNN Inference Result - LIT Dataset.....	48
[Table 18] CNN Inference Result - Private Dataset	49

Two-Step Load Identification Method for NILM System Based on Generative Adversarial Network and Convolutional Neural Network

Afifatul Mukaroh¹

Department of Information Convergence Engineering
The Graduate School
Pusan National University

Abstract

Energy consumption in the world is increasing. Consequently, lack of energy may occur, especially when demands of power electricity is higher than the available energy. In such way, energy conservation becomes one of concerns. Non-Intrusive Load Monitoring (NILM) is a system that correspondingly contributes to energy conservation. This is because power usage of each electricity appliance can be monitored using NILM. Through monitoring, the awareness of user about their power consumption can increase, then power consumption can be reduced. Unfortunately, there are still several issues in NILM. One of them is load identification across complex background load. Identifying a turning-on appliance that operating simultaneously across other loads becomes a challenge. This is because the other loads could be noises that reduce the load identification model performance. Therefore, we proposed two-step load identification methods in this thesis. First step is using Generative Adversarial Network (GAN) to denoise other appliances which are operating as background. Second step is using Convolutional Neural Network (CNN) to classify which appliance belong to the turning-on load. The approach was experimented using public dataset from LIT and private dataset collected in Lab. Experimental results demonstrate remarkable performance of the proposed method.

¹ The author of this thesis is a Global Korea Scholarship scholar sponsored by the Korean Government.

I. Introduction

Energy conservation becomes one of big issues in the world these days. This is because the power consumption is always increasing as the growth of population which also always increase. Consequently, high numbers of electricity demands cannot be avoided. Further, the possibility risk of lack electricity could arise if power demand is higher than supply. Various ways from different study fields have been conducted to address this issue. One of those is related efficient energy monitoring. By monitoring, users can be aware about how many powers consumption they spent, so that they can reduce it till approximately 15% [1, 2].

There two types of energy monitoring systems, Intrusive Load Monitoring (ILM) and Non-Intrusive Load Monitoring (NILM) [3]. ILM or NILM is a system to monitor appliance's power usage in household or industry. The number of sensors used is the main difference between them. ILM requires many sensors installed on the system, depending on the number of appliances. In contrast, NILM only installs one sensor in the system regardless the number of appliances. Thus, in term of cost, NILM is more preferred.

But because it uses one sensor only, there are many ways that the load is mixed among each appliance signals. That is why, NILM needs to be researched more in term of algorithm or mathematics approach. From those, we divided this section into four subs: Research Motivation, Related Work, Thesis Contribution, and Thesis Outline.

A. Research Motivation

Non-Intrusive Load Monitoring (NILM) has a role in load identification, where an appliance that has just turned on can be recognized. Load identification is classification task which map load signal data to a specific appliance name [4, 5]. In advanced application of NILM (e.g. demand response), load identification is commonly implemented for remote household control in smart grid [2]. For example, when the power consumption is overload, NILM should identify which appliance causing it, then

decides to turn off that appliance or not. That process should be done fast and automatic so that efficient microgrid set-up can be achieved [6].

Because load identification should be done automatically, it should only utilize short sample window of load signal and the execution time should be also short. Along with that, we utilize a concise window size signal of appliances as a load identification input. Therefore, it can shorten the waiting time to receive event information for users. Unfortunately, short period electrical signal features are easily submerged with background loads [3]. Background load can be noises in target load which may reduce the identification performance itself. Based on that problem, we are motivated to develop two-step load identification method. First step is by focusing to denoise that background load that mixed with target load using Generative Adversarial Network (GAN). Second step is by performing identification on the denoised load using Convolutional Neural Network (CNN).

B. Related Work

Several researches have been conducted to identify load. Researches by [2, 4, 7, 8] have succeeded to perform load identification, but their proposed method did not utilize short signal as the load identification input. Researches [9] have also succeeded to perform load identification and only utilized short signal as input, but the author did not consider the load that mixed with other load. Research [3] has succeeded to perform load identification, used short signal, and consider combined or mixed load. However, it has lower accuracy on some appliances having similar load between mixed load and background load [10]. Details about related work is presented in [Table 1].

[Table 1] Previous Research

No	Author	Method	Strength	Drawback
1	Zaidi, 2010 [7]	The author used Hidden Markov Models (HMM) to identify load in Microgrid's demand response.	Load identification was successfully performed. Based on the identified load, the inessential load is turned off.	Feature was taken from 2 minutes window. Thus, long waiting is required before identifying the loads. Low accuracy result is also still obtained by some appliances.

No	Author	Method	Strength	Drawback
2	Lin, 2014 [4]	The author used Ant Colony Optimization (ACO) method to identify load.	The proposed method can identify well although the target load is mixed with background load.	The input window size depends on transient period, which on some appliances is quite long. Besides, the accuracy is also still low.
3	Chang, 2014 [8]	The author used Parseval's theorem to preprocess power signal feature. Then for load identification method, author used Backpropagation Neural Network.	The number of features can be reduced. Thus, it has low execution time.	The input window size depends on transient period, which on some appliances is quite long. Moreover, it does not consider combined load.
4	Meehan, 2014 [2]	The author proposed two-step classification method to identify load. First step is classifying the load into 2 types based on the linear and reactive factor. Second step is classifying again into specific load appliance using Naïve Bayes.	The step one and step two of the proposed methods has good accuracy. Moreover, it identifies combined load.	It took 15 seconds of transient signal as the input; thus, long waiting is required.
5	Gebbe, 2019 [9]	The author used Support Vector Machine (SVM) method for load identification.	The feature was taken from short period signal (sixth period or 20-120 milliseconds after a turn-on event). It also has high f1-score.	This research only compromises classification problem without considering combination load. In other words, it only extracted features from single load.
6	Wu, 2019 [3]	The author proposed an architecture for load identification called the Concatenate Convolutional Neural Network (concatenate-CNN).	The features that is used as load identification input is only 7 seconds window size of transient signal. It also identifies combined load.	This approach has low accuracy to identify mixed load that is similar with the background load. [10]

C. Thesis Contribution

To overcome previous research's drawbacks, we propose two-step load identification method for NILM system based on Generative Adversarial Network and Convolutional Neural Network. This method utilized short period signal as input and experimented in combination load, but it maintains high accuracy. In details, the contributions are described as follows.

- a. Generative Adversarial Network (GAN) was utilized as first-step method to denoise background load by generating noise distribution, then extracting clean target load using this noise distribution.
- b. Convolutional Neural Network (CNN) was developed as second-step method to classify appliance based on generated clean target load by GAN.
- c. The proposed method was implemented and evaluated using augmented public dataset from LIT and private dataset collected in Lab, which contains combination load and only short period signal taken as input feature.

D. Thesis Outline

The rest of the thesis talks about background theory in section 2, proposed method in section 3, implementation and experimental result in section 4, followed by conclusion in section 5.

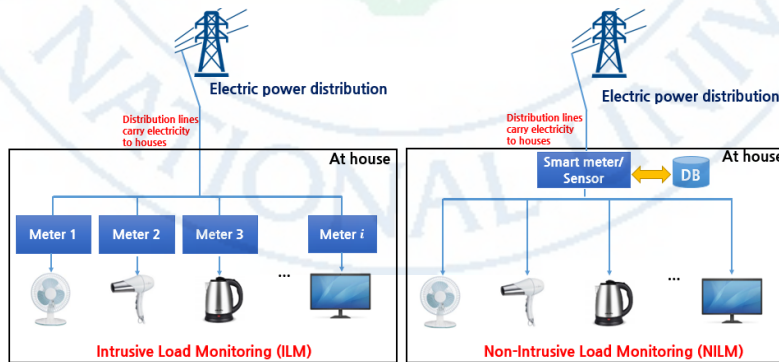
II. Background Theory

In section 2, there are three components explained: NILM, GAN, and CNN. Detail background theories about those components are described in following subsections.

A. NILM

In general, load monitoring is estimating and acquiring the load measurement in an electricity system. This load monitoring determines the status of an appliance and how many it consumes so that the behavior of individual loads from the entire system can be read [11]. Based on the approach in the monitoring, there are Intrusive Load Monitoring (ILM) and Non-intrusive Load Monitoring (NILM).

In ILM, each appliance is attached with a meter device or single sensor to monitor the activity of each appliance. The word “Intrusive” in ILM means that the metering device is located near the appliance that is going to be monitored [11]. Therefore, more appliance requires more metering device. It what makes ILM is expensive and hard to install, expand, or maintain. In contrast, NILM (Non-Intrusive Load Monitoring) is only installing a metering device at the panel level which infers what appliances are being used. Hence, this approach is cheaper and easier installation than ILM. [Figure 1] illustrates the difference between ILM and NILM.

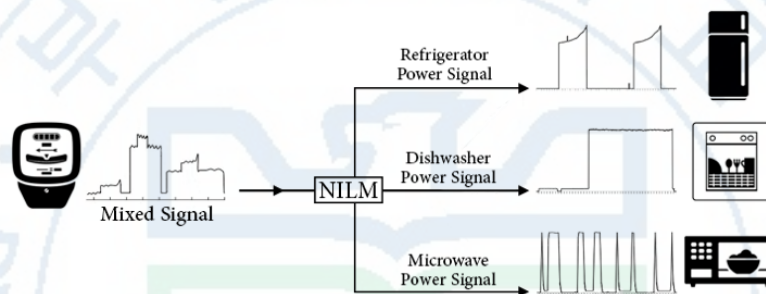


[Figure 1] Illustration of ILM and NILM difference

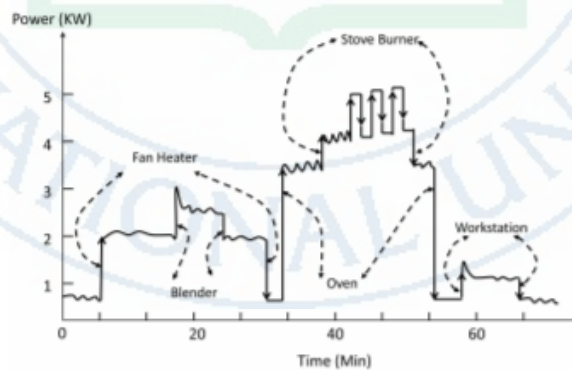
NILM was firstly proposed by Hart [12]. His research is able to detect ON-OFF event of appliances along with the interval time between them, then the power

consumption of the individual appliance is estimated. Basically, occupant can monitor each appliance usage in a home using NILM, although it is only using one metering device that being installed in energy entrance. NILM expected goal is to make monitoring becomes easier so that occupants may control their electricity usage. Further, it may lead to energy efficiency and reduce energy consumption.

Unfortunately, the power consumption of some active (status ON) appliances in NILM is aggregated because the metering device used in NILM is only one. Power Disaggregation is needed to monitor each appliance. [Figure 2] and [Figure 3] illustrate the aggregated Power.



[Figure 2] Illustration of Aggregated Power [13]



[Figure 3] Illustration of Aggregated Power [14]

1. NILM Objective

In general, NILM has three main objectives. Those are event detection, load identification, and load disaggregation. The description about those objectives are explained in subsections as follows.

a. Event Detection

Event detection can categorize the load whether it is in steady state (ON/OFF) or in transient state. It can be done by analyzing the power change of the signal [14]. Event/Load Detection also includes the detection of transient response time (how long the transient occurred) and what time the transient occurred. Some literatures conduct research related event detection. For example, a change of active power measurement from a high to low value or low to high value can identify whether the appliance is being turned ON or OFF [15]. Besides that, using shape, size, duration, envelope, and harmonics of the transient waveforms can also be used as method for event detection [15–17].

b. Load Identification

The goal of load identification is to identify which appliance is the source of transient event, either it is ON or OFF [3]. Load identification used to take uncompleted operation cycle of appliance. For example, it only takes the transient-state process of either ON or OFF events. Load identification also only concerns about how to obtain operation status of appliances [18].

Advanced applications of smart grids commonly leverage load identification, such as implementing demand response for remote appliance control [19]. Demand response identifies the type of operating load, then deciding whether to turn off or not when the peak of load consumption is reached. The process of load identification should be done automatically to have an easier microgrid set-up [7]. Several supervised learning methods based on transients' features are mostly utilized for load

identification [3, 8, 9]. Further, the result of load identification can also be beneficial to enhance the energy disaggregation task in NILM.

c. Load Disaggregation

Load disaggregation estimates the power consumption by individual appliance from a single aggregated power using metering devices [20]. Load disaggregation used to proceed complete operation cycle of load from ON to OFF. Some researchers even sometimes ignore ON and OFF event. They performed load disaggregation result at hour level of power consumption [19].

Specifically, the problem of energy disaggregation can be formulated as follows [20]: Given the sequence of aggregate power consumption $X = \{X_1, X_2, \dots, X_T\}$ from N active appliances at the entry point of the meter at $t = \{1, 2, \dots, T\}$, the task of the NILM algorithm is to infer the power contribution y_t^i of appliance $i \in \{1, 2, \dots, N\}$ at time t , such that at any point in time t ,

$$X_t = \sum_{i=1}^N y_t^i + \sigma(t) \quad (1)$$

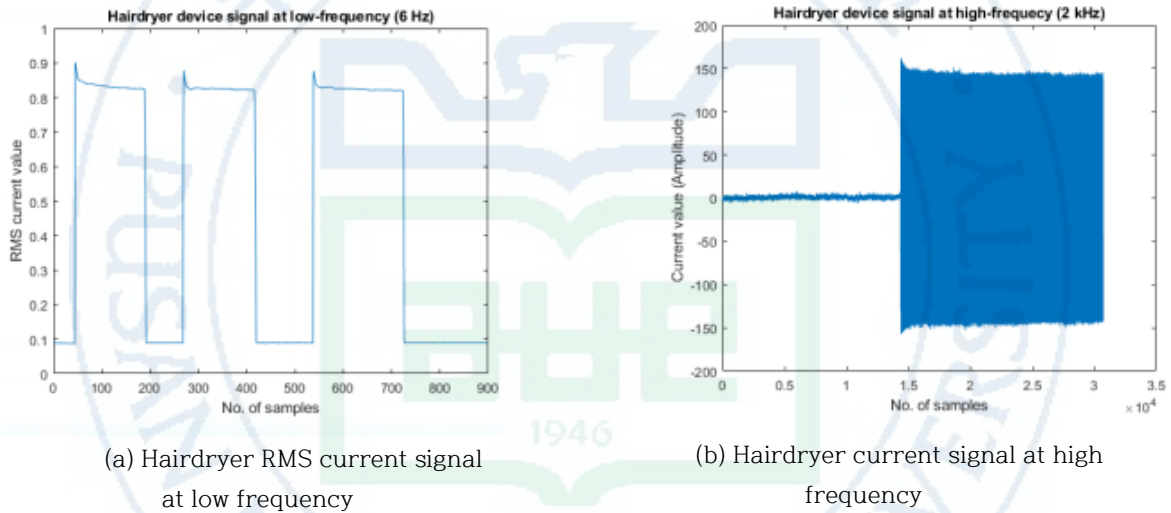
where $\sigma(t)$ represents any contribution from appliances not accounted for and measurement noise. Methods that have been used for load disaggregation are HMM (Hidden Markov Model), deep learning, and Graph Signal Processing [20].

2. Features in NILM

NILM features can be obtained through data acquisition process. Data Acquisition is collecting aggregated load measurement at an adequate sampling rate so that distinctive load patterns can be identified. Based on the sampling rate, type of smart meter is divided into Low-Frequency Energy Meters and High-Frequency Energy Meters [20]. The term of sampling rate means how many samples per second (in measurement unit Hz). For example, if we collect the home appliances at 6 Hz, it means

the sampling rate data is 6 samples per second. If we collect the household appliances at 2 kHz, it means the sampling rate data is 2000 samples per second.

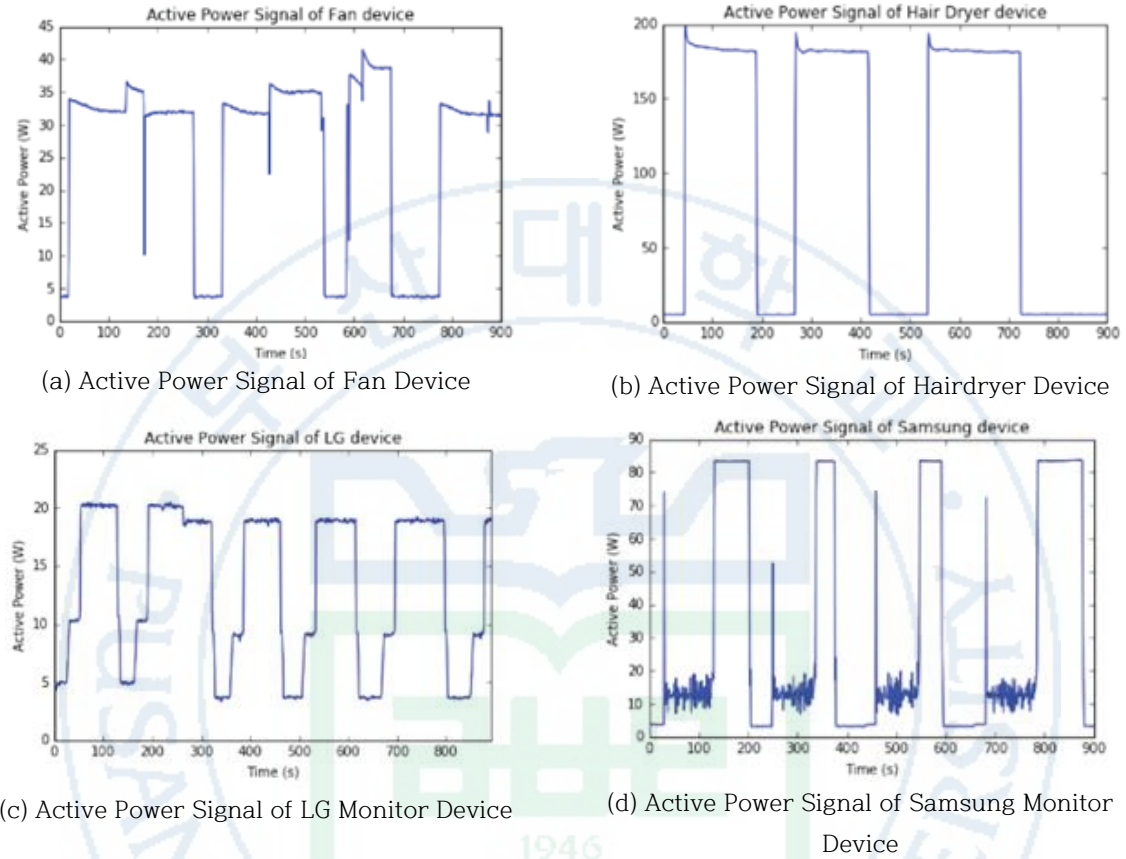
Low-Frequency Energy Meters measures in small sampling rate (less than 1Hz) [20]. It computes traditional power metrics, like real power, reactive power, Root Mean Square (RMS) voltage dan RMS current. It is also cheap in installation. Meanwhile, High-Frequency Energy Meters measures in high sampling rate (10 MHz - 100 MHz) [20]. It is good to capture transient state and custom-built. The feature that is going to be extracted also can be tailored (custom-made) even though it's more expensive than low sampling rate [20]. The difference between low frequency and high frequency can be seen in [Figure 4].



[Figure 4] Difference between low frequency and high frequency

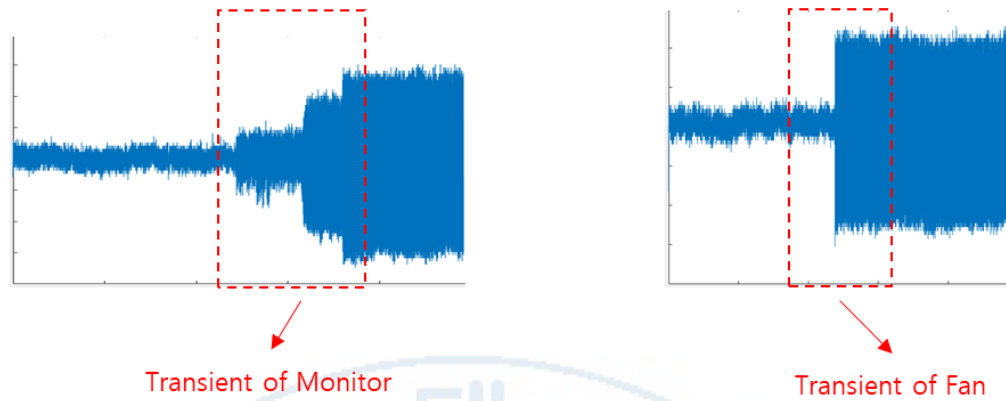
There are two types of appliance signatures used by NILM : steady state feature and transient-state feature [20]. Steady state feature of a signal is when an appliance has the more sustained changes in power characteristics after it is turned ON. In other words, steady-state signal is the stationary data of load's operating. Steady states feature includes active power [21, 22], reactive power [23], current [24], current and voltage waveforms [25], and others. The acquisition of steady-state signature does not demand high-end metering devices; thus, it is the most used features at low frequency in the literature [20]. Previous works [21, 26, 27] use steady state of real-power for disaggregation, while [24] criticized in his paper that current is a more effective steady-

state feature for energy disaggregation problem instead of real-power. [Figure 5] illustrates steady state. Based on the figure, each appliance has different active power pattern, active power maximum number, and so on.



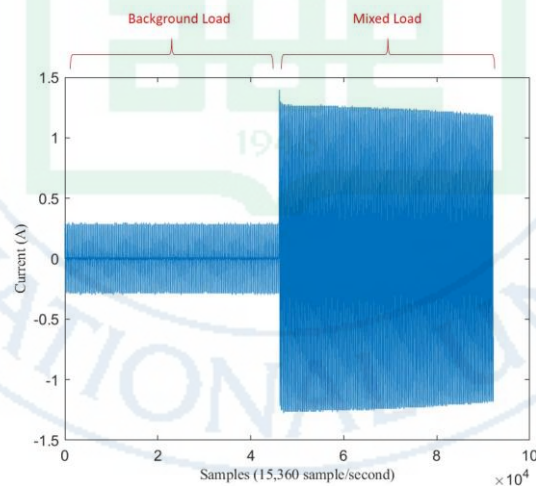
[Figure 5] Illustration of steady state

Transient state feature of a signal is when an appliance has just turned on and there are short time fluctuations in the signal before settling into a steady-state value [20]. In other words, transient data is obtained before the load obtains stationary data. Transient-state feature is unique. It can distinct each appliance by extracting features like duration, harmonics, size, and shape of the transient [14]. However, to obtain the signal uniqueness high sampling rates of data acquisition is required [23, 28]. Longer monitoring time is sometimes also required to capture all operation cycles, because each appliance has different transition-state duration. [Figure 6] illustrate transient state. The figure shows that each appliance has different shape, size, and duration of its transient state.



[Figure 6] Illustration of transient state

In transient state signal, a load before the transient event is called background load. If there is no background load, a turning-on appliance signal is considered as single load. In this case, target load is interpreted as equal to single load. But if the load contains background load, the target load is going to be mixed with background load. The mix between target load and background load is called mixed load. [Figure 7] illustrate background load and mixed load in a sequence signal. We took Mixed Load of LED Lamp and Fan for example in this figure.



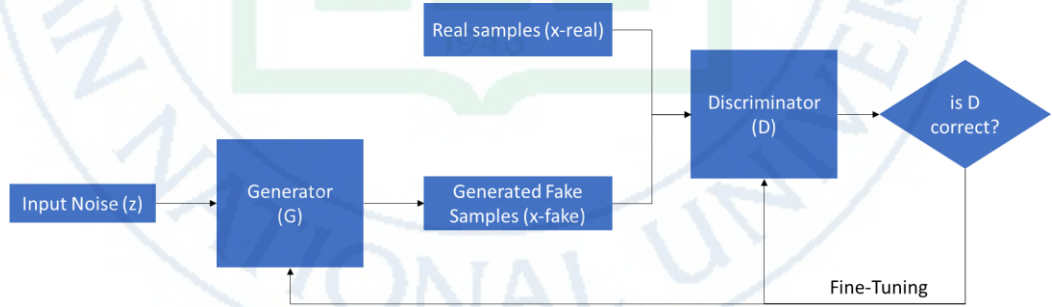
[Figure 7] Illustration of Background Load and Mixed Load [10]

B. GAN

GAN or Generative Adversarial Network was first proposed by [29]. It consists of two adversarial networks: Generator and Discriminator. These two models compete each other in a form of a game setting [30]. The discriminator model's primary goal is to determine fake from real data. While the generator model's primary goal is generating artificial data that can pass the discriminator successfully [30].

The learning model in GAN is the generator. Initially, generator is likely to produce low or even completely noisy data that does not reflect the real distribution or the properties of the real data. The model starts taking some noise, usually Gaussian noise and produces an image formatted as a vector of pixels. The generator must learn how to trick the discriminator and win a positive classification (produced image classified as real).

During the training, generative model G captures the data distribution, and a discriminative model D estimates the probability that a sample came from the training data rather than G . Training Procedure of GAN is illustrated in [Figure 8].



[Figure 8] GAN Training Illustration

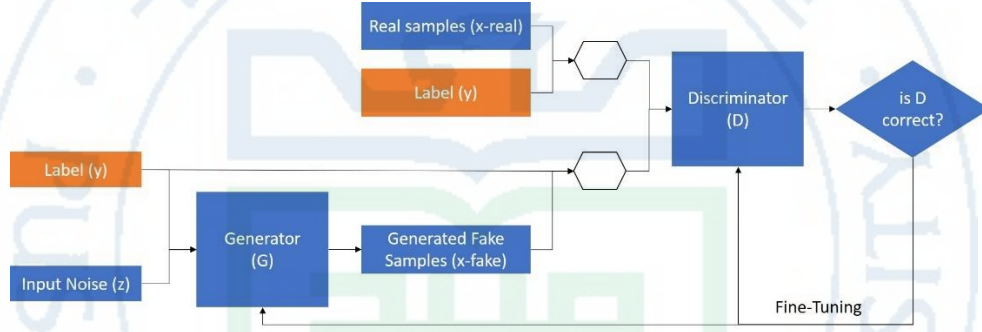
G and D are both trained simultaneously: the parameters for G is adjusted to minimize $\log(1 - D(G(z)))$, while parameters for D is adjusted to minimize $\log D(X)$. Both models are following the two-player min-max game with value function $V(G, D)$: [31]

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (2)$$

To learn a generator distribution p_g over data x , the generator builds a mapping function from a prior noise distribution $p_z(z)$ to data space as $G(z; \theta_g)$. And the discriminator, $D(x; \theta_d)$, outputs a single scalar representing the probability that x came from training data rather than p_g [31].

1. Conditional GAN

Conditional GAN or cGAN is basically extended from GAN. cGAN is very similar to GAN, except both the generator and discriminator are conditioned based on extra information, y . y could be anything, such as class labels or data from other process. The conditioning can be performed by feeding additional input layer (y) into both discriminator and generator. Training Procedure of cGAN is illustrated in [Figure 9].



[Figure 9] Conditional GAN Training illustration

In cGAN, the input noise $p_z(z)$ and y of generator are combined in hidden data representation. The representation is flexible considering the adversarial training framework. Meanwhile, x and y of discriminator represents as input and discriminative function. The objective function can be interpreted as two-player minimax game. It would be formulated as Equation (3) [31].

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x|y)] + E_{z \sim p_z(z)} [\log(1 - D(G(z|y)))] \quad (3)$$

2. Pix2pix GAN

Pix2Pix GAN was firstly proposed by Isola [32]. Pix2Pix GAN is basically a type of cGAN, where the output image generation depends on the input or source image.

Meanwhile, the discriminator has a role to determine whether the target is a plausible transformation of the input image. Thus, source image and the target image (label) must be provided in Pix2pix GAN.

The objective of Pix2pix GAN is combining the objective of conditional GAN and Loss L1 distance. Loss L1 distance is added because it encourages less blurring [32]. Besides that, It is beneficial to mix the GAN objective with a more traditional loss [33]. This way, discriminator's task remains unchanged. Moreover, generator will not only fool discriminator but also will attempt to be as close as possible with the ground truth output in an L1 sense. This Pix2pix GAN objective is presented in Equation (4), with λ as the weight on the L1 objective towards the GAN objective. Equation (5) formulates the conditional GAN and Equation (6) formulates the Loss L1 distance.

$$G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G) \quad (4)$$

where,

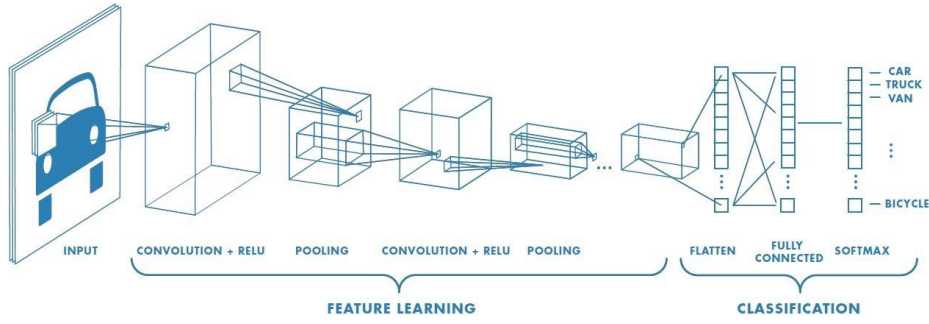
$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y} [\log D(x, y)] + \mathbb{E}_{x,z} [\log(1 - D(x, G(x, z)))] \quad (5)$$

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} [\|y - G(x, y)\|_1] \quad (6)$$

C. CNN

CNN is one of deep learning method that leverages mathematical linear operation between matrixes which is called convolution. CNN is built from sequence of layers, such as convolutional layer, pooling layer, and fully connected layer. In machine learning issues especially applications that relate to image data, CNN has an outstanding performance.

CNN has two components; those are feature extraction part and classification part [34]. [Figure 10] shows the CNN framework.



[Figure 10] Illustration of CNN framework

1. Feature Extraction Part

The goal of feature extraction is to extract the most representative feature of the input image so that the size of input image can be reduced. Feature extraction consists of several series process, such as convolutions and pooling operations.

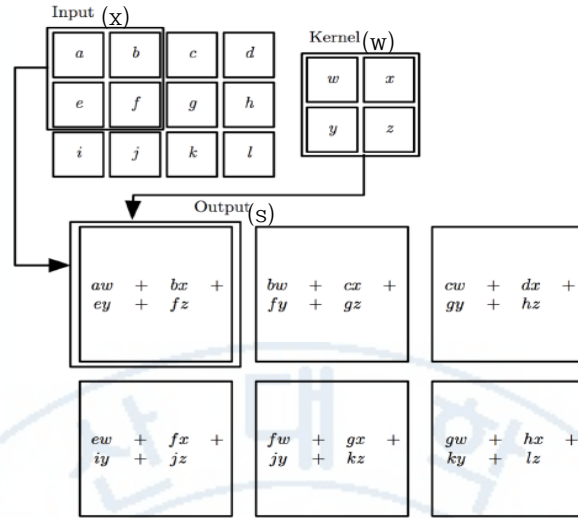
a. Convolution

Convolution is the main block in CNN. It is performed by multiplying input data with a filter or kernel, which in the end it will yield a feature map. This feature map is unique. Convolution is executed by sliding the filter pixel by pixel of input image. At every position of pixel, the multiplication is performed. After that, sum is performed till the feature map is obtained.

Illustration of convolution operation can be seen in [Figure 11]. It is formulated as Equation (7) where x is input image, w is kernel, the result s .

$$s(i,j) = \sum_m \sum_n x(i+m, j+n)w(m,n) \quad (7)$$

In CNN cases, an activation function is needed so that all the convolution result can pass through this. The activation function could be Tanh activation function, Softmax activation function, ReLU activation function, or others.



[Figure 11] Convolution Illustration

b. Stride

Stride is how many step the convolution filter moves from one position of pixel to other [34]. The default size of stride is 1, meaning the filter slides by only one pixel. Larger steps will occur if stride size is bigger. But feature map may be more shrinking because of it. To prevent this, a padding is needed.

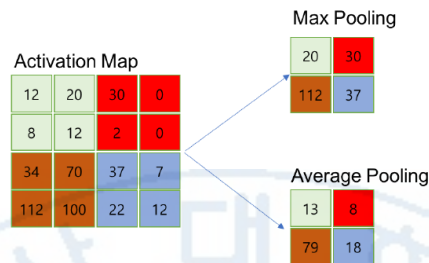
Padding is a layer of zero-value pixels which is added to surround the input in order to make the feature map not shrink. Padding also keeps the spatial size constant after convolution, which mean the input and the result of convolution have same size.

c. Max Pooling

Max Pooling mostly follows convolution layer. The goal of pooling is to reduce the dimensionality continuously so that the memory and computation complexity can be minimized. Further, pooling can shorten the training time and also control overfitting [34].

Types of pooling are: max pooling, which extract the biggest value from the feature map; average pooling, which extract the average value from the feature map; and sum pooling, which sum of all values in the feature map. Among three of them,

max pooling is mostly used. Pooling decreases the feature map size and also keeping the significant information. [Figure 12] illustrate max pooling and average pooling illustration.

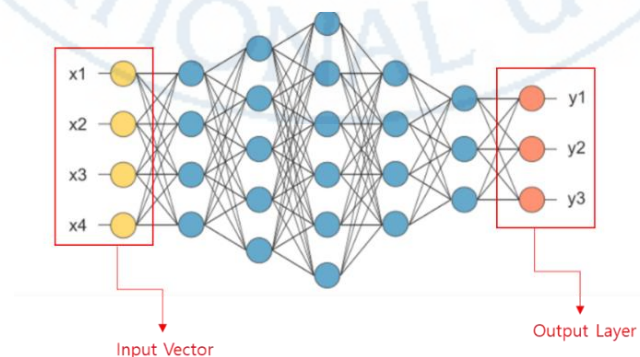


[Figure 12] Max Pooling and Average Pooling Illustration

2. Classification Part

Classification part is when a fully connected layers on the top of extracted features to classify the input [34]. For instance, the classification part will assign the probability for each image input about what the algorithm is predicting it. Classification part is built from several fully connected layers. This fully connected layers mostly only accept 1-dimensional data. Thus, if the input is 3-dimensional data, it must be flattened.

Fully connected layers can be considered as the last layer of CNN. Hence, all previous layer activations are connected to every nodes of fully connected layer. From that, this classification part has the same principle as a regular Artificial Neural Network (ANN). [Figure 13] illustrates fully connected layer.



[Figure 13] Fully Connected Layer Illustration

III. Proposed Method

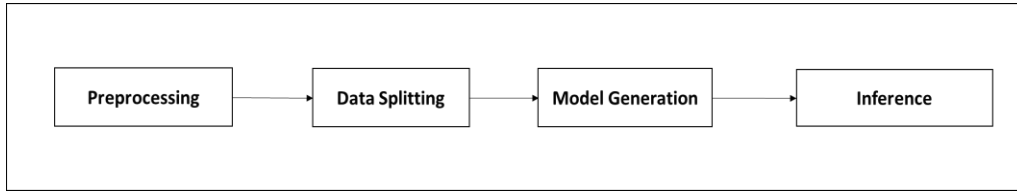
In section 3, there are five components explained: General Framework, Preprocessing, Data Splitting, Model Generation, and Inference. Subsections below describes those components.

A. General Framework

In this research, we attempt to enhance the identification of load, especially load that combined with other load using short period times of signal. As we mentioned in section 1, short period signal makes target load easily mix with background load [3]. The more noise will be added to the load, if more appliances are simultaneously operating as the background [2].

Suppose target load that is going to be identified is expressed as T , and the identification result (appliance class) is expressed as A . Load identification aims to solve a function $F : T \rightarrow A$ (a mapping from T to A). The issue in load identification, especially when the sample target load is taken in short window, is the load that is going to be identified is mixed with background load B . Suppose target load T that is mixed with background load B is expressed as Mixed Load M , then $F : M \not\rightarrow A$. Based on that reason, in this thesis we proposed two-step load identification method to solve that issue. First step is finding a function model that satisfies $G : B, M \rightarrow T$ based on GAN, and second step is finding a function model that satisfies $F : T \rightarrow A$ based on CNN.

In general, our proposed method consists of 4 main process as shown in [Figure 14], starting from preprocessing, data splitting, model generation, and then inference. Preprocessing is a proses to prepare the raw dataset so it can be processed in the next step. Data Splitting is a process to divide which dataset for training and which dataset for testing. Model Generation is a process to generate and choose the best model based on iterative training and testing steps. Inference is a process to infer the trained models with unseen data, so that the performance of the proposed method can be measured.



[Figure 14] Main Process of Proposed Method

B. Preprocessing

We performed preprocessing for mixed load dataset and single load dataset. The signal from each dataset is transformed into spectrogram image using STFT (short term Fourier transform) approach. Then, as it becomes image dataset, pixel of its image is used the features. Some references [36–38] have shown promising result by using pixel from spectrogram image as the feature of recognition task. The spectrogram is also transformed from a waveform signal. The uniqueness of color distribution in spectrogram image can be useful as feature map in identification task.

In single load dataset preprocessing, data augmentation is performed at signal dataset because the amount of dataset is small. Data Augmentation is performed by adding gaussian noise to the signal. After Data augmentation, the next process is transforming it into spectrogram. The input is 3 seconds single load data after turning on and the output is image RGB channel with size 256x256, which read as matrix size 3x256x256. This matrix is paired with appliance name as label for CNN-based load identification training in the next step. It can be seen in [Figure 15].

In mixed load pre-processing, there is another process to estimate noise distribution label, that is used in GAN-based noise generation training. Estimating noise distribution itself is inverse problem from equation (8) [38].

$$X = \hat{X} + E \quad (8)$$

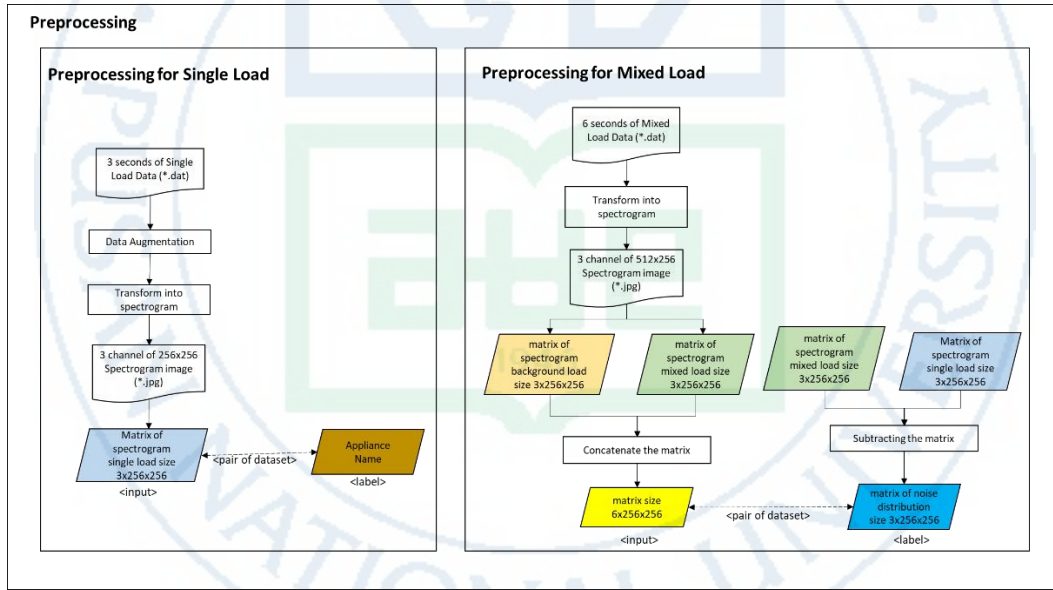
Where, $\hat{X}$ is the clean image, X is the noisy observation, and E is noise distribution E . Image denoising refers to a task of recovering clean image $\hat{X}$ from its noisy observation X which is containing noise distribution E [38]. In this case, mixed

load is the noisy observation (because it has been mixed with background load) while single load is clean image (because it is not mixed with any background load).

Suppose spectrogram of background load is denoted as S_b , spectrogram of mixed load is denoted as S_m , and spectrogram of single load is denoted as S_s , then based on Equation (8), we can calculate noise distribution E with Equation (9).

$$E = S_m - S_s \quad (9)$$

Thus, as can be seen in [Figure 15], label of noise distribution is obtained by subtracting spectrogram of mix load (S_m) with spectrogram of single load (S_s). Meanwhile the feature that is used for input is concatenate between mixed load and background load, expressed as $[S_b, S_m]$.



[Figure 15] Preprocessing

In preprocessing, there are two main process: data augmentation and spectrogram transformation. For data augmentation, we use Addictive White Gaussian noise (AWGN). For spectrogram transformation, we use Short-Time Fourier Transform (STFT).

1. Data Augmentation

Data augmentation defines as a process of generating new samples by transforming the data. The aim of data augmentation is to increase the accuracy and robustness of classifiers [39] and dealing with the low number of training dataset [40]. In research by [40] used gaussian noise to augment signal. In his research, he analyzed that adding noise to signal is better approach for augmentation in signal. Unlike image processing, which mostly uses geometric transformation as augmentation approach. This is because compared with an image, a continuous signal changes over time.

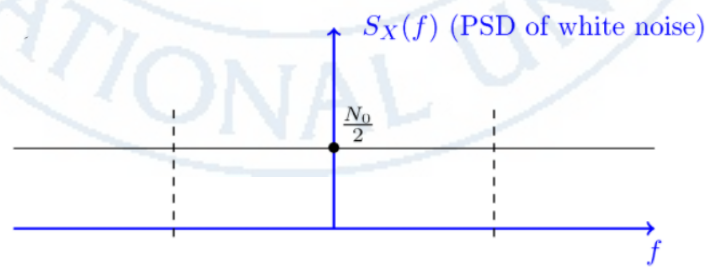
Data Augmentation was performed by adding Addictive White Gaussian noise (AWGN) to raw signal. AWGN is one of methods to generate noise with following aspects [41]:

a. Addictive

The noise is additive if the received signal is equal to the transmitted signal plus noise, where the noise is statistically independent of the signal [42].

b. White

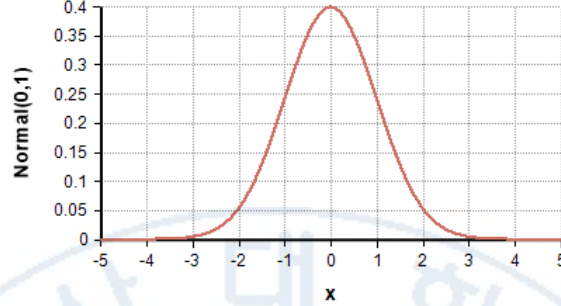
The random process $X(t)$ is called white noise if Power Spectral Density (PSD), $S_x(f)$, is constant for all frequencies. By convention, the constant is usually denoted by $N_0/2$ [43]. [Figure 16] illustrate the PSD of white noises.



[Figure 16] PSD of White Noise

c. Gaussian

The noise samples have Gaussian probability distribution with a zero mean, $\mu_X = 0$. [Figure 17] illustrates Gaussian Probability Distribution.



[Figure 17] Gaussian Probability Distribution

2. Spectrogram Transformation

The spectrogram images dataset was transformed from signal waveform data using Short-Time Fourier Transform (STFT). STFT is several Fourier transforms that is performed sequentially on windowed signal [44]. The basic Fourier transform result is only frequency information over all completed signal time. Meanwhile, STFT result is frequency information which is time-localized based on the specific windowed signal.

STFT pair can be formulated by Equation (10) and (11) [44].

$$X_{STFT}[m, n] = \sum_{k=0}^{L-1} x[k]g[k-m]e^{-j2\pi nk/L} \quad (10)$$

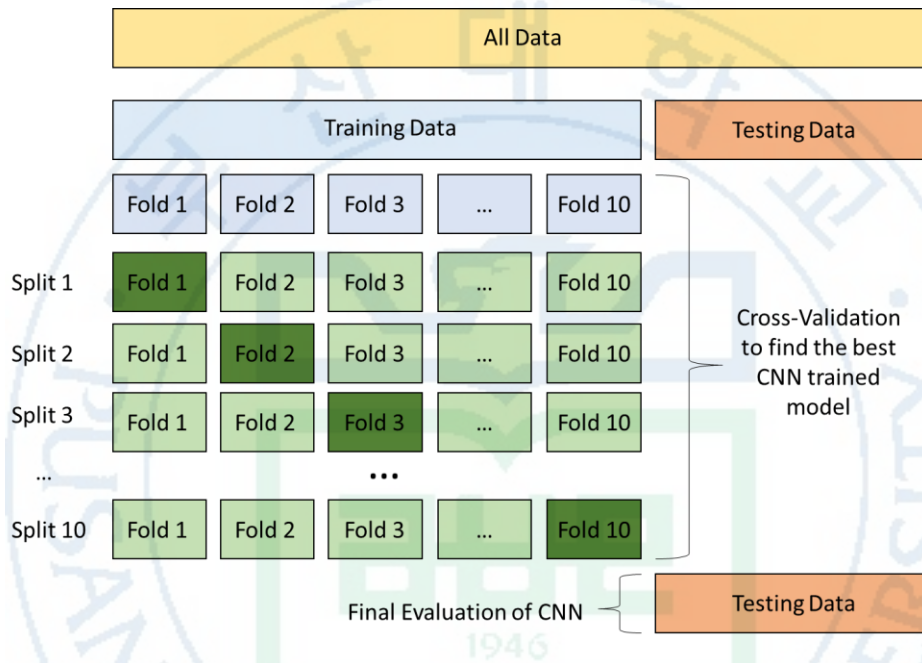
$$x[k] = \sum_m \sum_n X_{STFT}[m, n] g[k-m]e^{j2\pi nk/L} \quad (11)$$

where $x[k]$ denotes a signal and $g[k]$ denotes an L-point window function. STFT of $x[k]$ can be interpreted as the Fourier transform of the product $x[k] g[k-m]$.

Spectrogram is the representation of the Power Spectral Density (PSD) from magnitude squared of STFT output [44]. If the PSD is higher, the spectrogram image will have darker color. If the PSD is lower, the spectrogram image will have lighter color [45].

C. Data Splitting

Data splitting for single load dataset is performed according stratified k-fold cross validation with parameter k is 10. Stratified k-fold is an advance type of k-fold cross validation. It avoids imbalance data splitting, because it sets the samples of each class has nearly the same percentage. Illustration of this k-fold cross validation is presented in [Figure 18]. This single load dataset is used for CNN training.

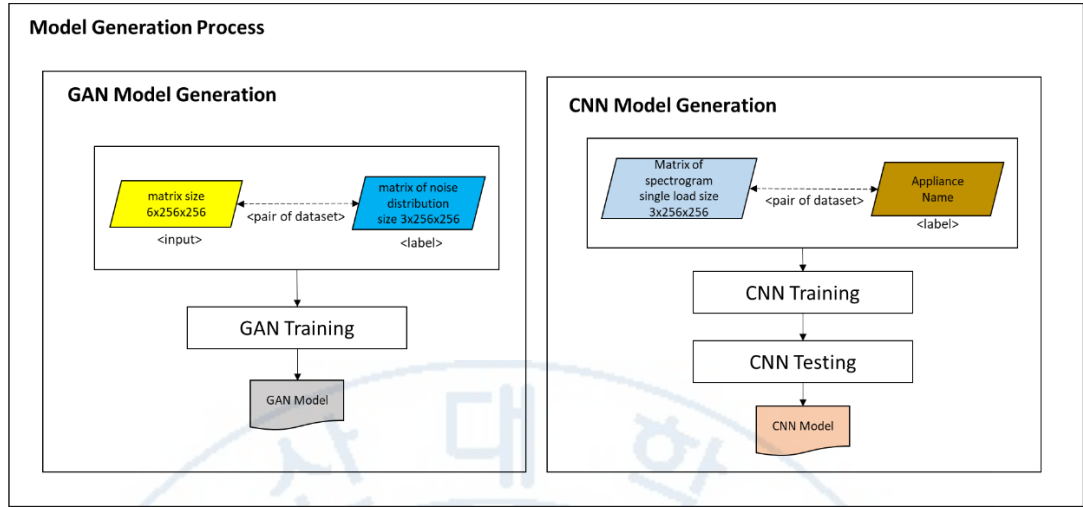


[Figure 18] Illustration Of 10-Fold Validation in Proposed Method

Data splitting for mixed load dataset is performed by dividing the data into 70% training data and 30% is inference data. This mixed load dataset is used for GAN training.

D. Model Generation

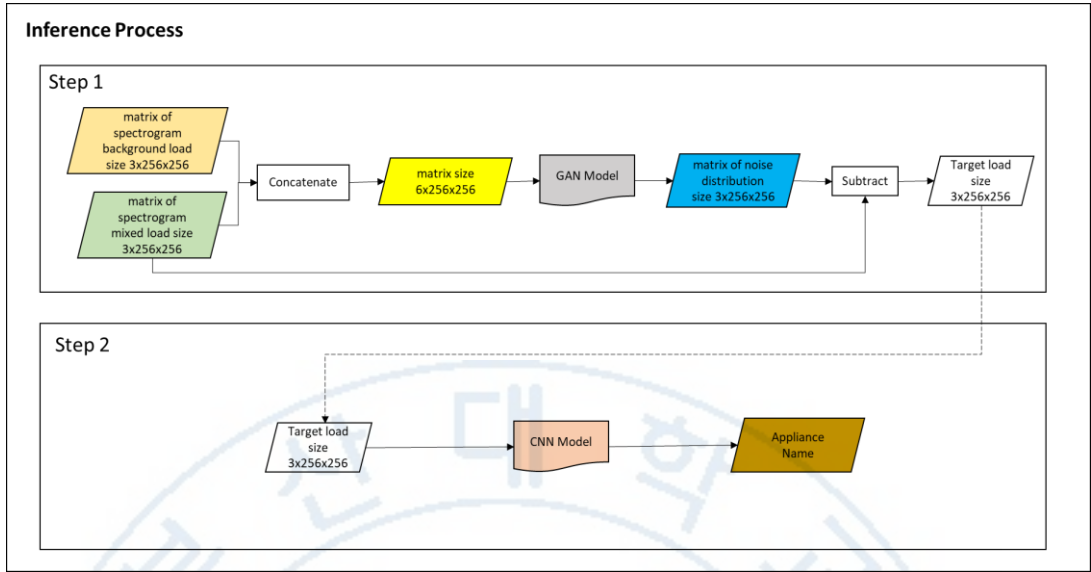
There are two model that needs to be generated—CNN model and GAN model. GAN model is to generate noise distribution, which then clean target load is processed using that noise distribution. Meanwhile, CNN model is to classify the turning-on load based on the generated clean target load from previous step. [Figure 19] illustrate the Model Generation in general.



[Figure 19] Illustration of Model Generation

E. Inference

Inference process is performed to know the performance of two-step load identification with unseen mixed load data. [Figure 20] illustrates the inference process. In step 1, the input is matrix of spectrogram background load that being concatenated with matrix of spectrogram. Then, using trained GAN Model, a matrix of noise distribution is generated. After that, based on equation (9) target load is obtained by subtracting mix load with noise distribution. In step 2, the target load is identified using trained CNN model to know which appliance cause the transient event.



[Figure 20] Illustration of Inference Process

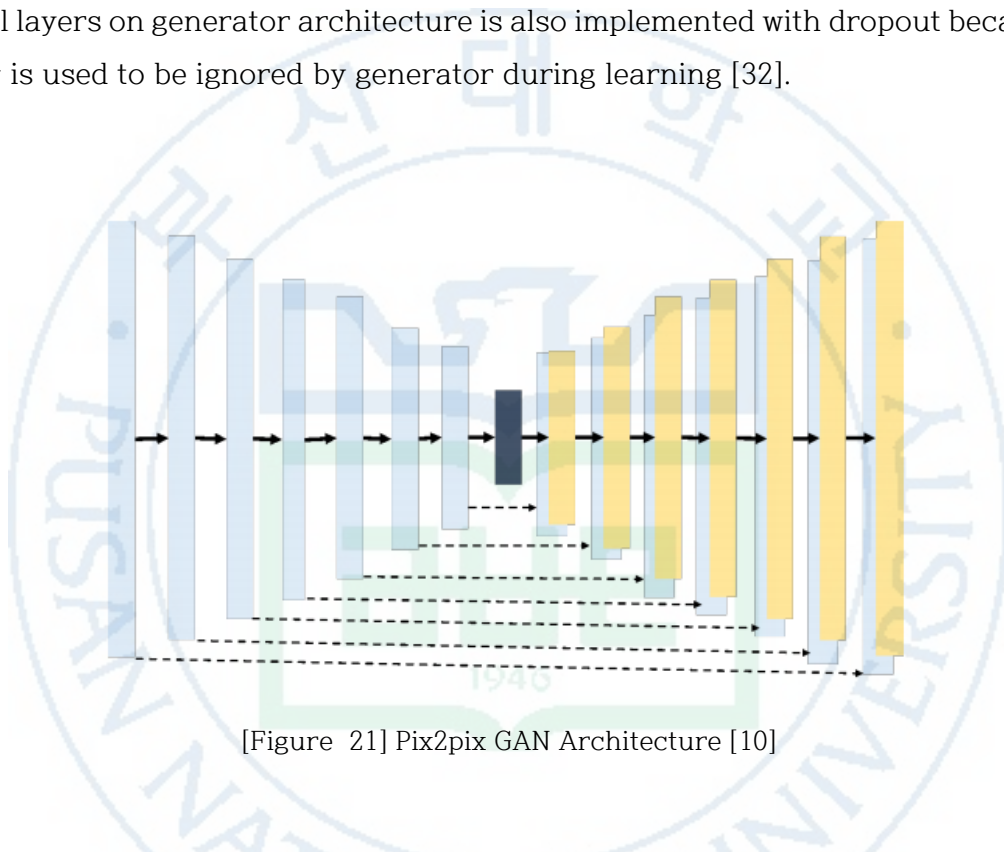
a. GAN Model Architecture

Our previous research of GAN as a model for denoising complex background load [10] is adapted to our thesis. We also utilized Pix2pix GAN in our thesis as the model architecture. As described in section 2, Pix2pix GAN training requires a pair dataset (source image and target image) because it is actually a part of conditional GAN. Therefore, we use preprocessed mixed load from Subsection 3.A.1 as the input.

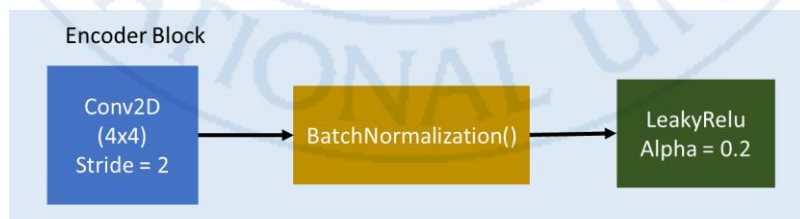
The architecture of our Pix2pix GAN is built from 7 encoder blocks and 7 decoder blocks. These blocks were connected with bottleneck layer. [Figure 21] shows the architecture of our Pix2pix GAN, where light blue block represents the encoder blocks, orange block represents the decoder block, and dark blue block represents bottleneck layer. The dot line in [Figure 21] represents skip connection. Skip connection is concatenation between the output of encoder block output and decoder block. It is processed by *concatenate layer* in decoder block. This way, the issue of bottleneck between encoder-decoder blocks can be minimized. Normally, the input that is passed through the layers will gradually down-sample. When it has down-sampled, there is likely lost information. Whereas all information from the input

feature map should pass through all the network layers. Thus skip connection is needed to connect the corresponding layers between encoder and decoder blocks [32].

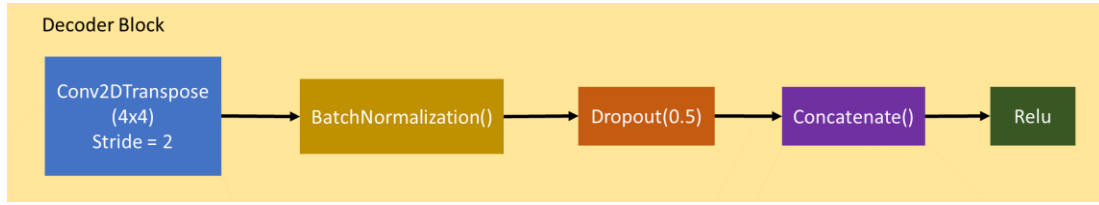
The Bottleneck Layer composes of *Conv2D* and *ReLU* . Encoder Block components is presented in [Figure 22]. It composes of *Conv2D*, *BatchNormalization*, and *Leaky ReLU*. Decoder block components is presented in [Figure 23]. It composes of *Conv2Transpose* , *BatchNormalization* , *Concatenate layer* , and *ReLU* Activation. Several layers on generator architecture is also implemented with dropout because the noise z is used to be ignored by generator during learning [32].



[Figure 21] Pix2pix GAN Architecture [10]



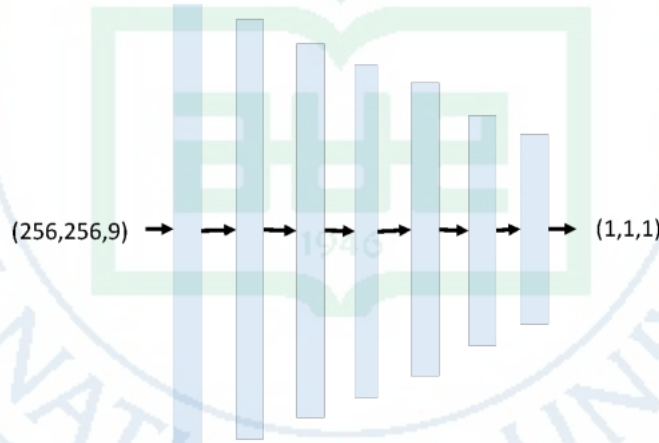
[Figure 22] Encoder Block Components [10]



[Figure 23] Decoder Block Components [10]

The image translation task in our pix2pix GAN is translating spectrogram image of 6 second transient loads into noise distribution. Therefore, we used 3 channels spectrogram image of background load that is concatenated with 3 channels spectrogram image of mixed load as the input or source image. Then, 3 channels of noise distribution are the output.

The architecture of Discriminator consists of 1 *concatenate* layer, 5 *Conv2D* layers with *Leaky ReLU* activation, and 1 *Conv2D* layer with *sigmoid* activation. [Figure 24] depicts the discriminator architecture.



[Figure 24] Discriminator Architecture of Pix2pix GAN. [10]

In our case, discriminator attempts to classify whether the generated noise distribution is plausible with the source image. Therefore, the input of discriminator is the source image (3 channels spectrogram image of background load that is concatenated with 3 channels spectrogram image of mixed load) and noise distribution

target (label) which is calculated from Equation (9). The output of Discriminator is patch image with size 1×1 . This 1×1 output patch has a meaning whether it is real or fake.

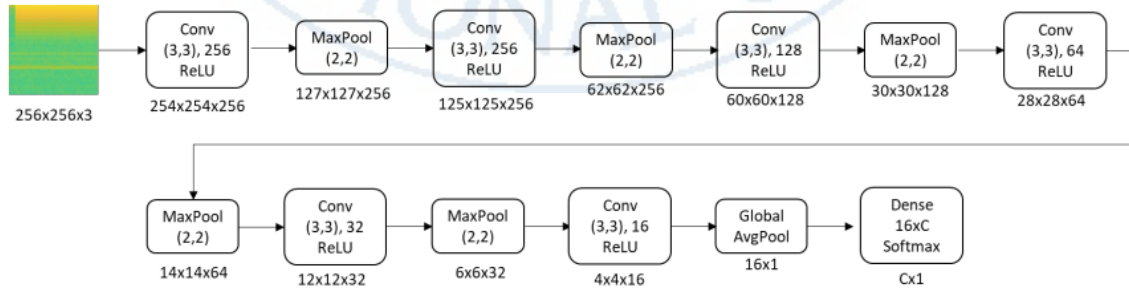
For Discriminator, we use binary cross entropy as loss function. We choose this function because basically discriminator is like CNN model with two class output (fake or real). Binary cross entropy is defined as Equation (12).

$$L = \left(\sum_{i=1}^n y_i \log a_i + (1 - y_i) \log(1 - a_i) \right) \quad (12)$$

where the variable n is the total number of features, variable a is the predicted output and variable y is the target output.

b. CNN Model Architecture

The CNN model architecture in this thesis is also adapted from our previous work [10]. [Figure 25] shows the CNN architecture of our proposed network. As mentioned in subsection 2C, CNN architecture comprises of two parts: feature extractor and classifier. The feature extractor part of our architecture consists of 6 convolution layers and five max pooling layers. The convolution layer is built from 16 to 256 filter convolution of kernel size 3×3 and stride 1. The max pooling layers has 2×2 sizes. A global average-pooling layer follows these layers to compress the feature map size. The classifier part of our architecture is built from a fully connected layer with $16 \times C$ dimension, with C as the number of classes. The total parameters of our proposed architecture are 989,321.



[Figure 25] Convolutional Neural Network (CNN) architecture [10]

This CNN architecture receives input of single load spectrogram image, with size 256x256 (RGB image). This single load spectrogram image is processed through a CNN network and the output is a specific class of appliance type.

We used Cross-entropy function for our CNN network. This function can be expressed as Equation (13).

$$L = \left(\sum_{i=1}^c y_i \log Z_i + (1 - y_i) \log(1 - Z_i) \right) \quad (13)$$

where y_i is the i^{th} element label y of C classes, and Z_i is the i^{th} element of output Z , which has function activation *softmax*. Function activation *softmax* can be represented as Equation (14).

$$Z = \text{softmax} (h_{\sigma}(S_s)) \quad (14)$$

where h_{σ} is the classifier function that map single load spectrogram image (S_s) to specific class of appliance. *Softmax* is the activation function.

IV. Implementation and Experimental Result

In section 4, there are six components explained: Environment Specification, Dataset, Data Preprocessing and Data Splitting, GAN Model Generation, CNN Model Generation, and Inference. Detail about those components is described in subsections below.

A. Environment Specification

The GAN model training was implemented in PC with specification of 8GB RAM, Intel Core i7 processor, and Intel HD Graphics 620. While the CNN model training and inference process were implemented in PC with specification of 32 GB RAM, Intel Core i7 processor, and GPU NVIDIA GeForce 1650.

The preprocessing was implemented using Matlab R2020a with its signal processing and image processing toolbox, while the rest of processes were implemented in Jupyter Notebook with Python 3 environment and Keras/Tensorflow library.

B. Dataset

We did experiments two times with two types of dataset-- public dataset and private dataset, which both of it are high-frequency dataset of transient current signal. Public dataset is dataset that available online for public freely, while private dataset is dataset that we collected in our Lab. LIT-dataset is used for experiments, while private dataset is used to confirm that the result of our experiment also work well for other dataset.

1. LIT-dataset

LIT-dataset was taken by Laboratory for Innovation and Technology in Embedded Systems[46]. The dataset was collected in high frequency sampling, which is 15,360. Label of each target load is also provided by LIT-dataset. Besides that, the load has various types scenario of background load. Each scenario was collected 16

times with different triggering angle. Some background loads compose of one operating appliance. Some others compose of more than one combination of operating appliance.

This dataset provides many appliances options, but we only chose 9 class appliances which can be seen in [Table 2]. We chose those nine types of because they have various combination of background load. The mixed load data was used to train GAN model, with total data is 656 samples. Details number of LIT-dataset can be seen in [Table 2].

[Table 2] Mixed Load Dataset from LIT-Dataset

Name of Target Load Appliance	Number of Data
LED Lamp	112
LED Panel	48
Fume Extractor	80
Motorola Phone Charger	80
Fan	32
VIAO Laptop	144
Drill Speed 2	64
Microwave Oven ON	48
Hairdryer	48
Total Data	656

Meanwhile, each appliance class of the single load dataset is only 16 samples, so that the total number is 144 samples. Since the amount of data is small, we also performed data augmentation. The total data after augmentation is 1,008 samples, with detail of each appliance data is presented in [Table 3].

[Table 3] Single Load Dataset from LIT-Dataset

Name of Target Load Appliance	Number of Data
LED Lamp	112
LED Panel	112
Fume Extractor	112
Motorola Phone Charger	112
Fan	112

Name of Target Load Appliance	Number of Data
VIAO Laptop	112
Drill Speed 2	112
Microwave Oven ON	112
Hairdryer	112
Total Data	1,008

2. Private Dataset

The private dataset consists of current dataset with 15,200 frequency sampling. The acquisition process conducted based on specific scenario, where an appliance was turned on across other operating appliances. Thus, mixed load dataset can be obtained. Each scenario was taken 5 times. The combination of loads can be two or three loads. And every turning-on event was marked to set the appliance label.

There are three appliances selected. Those are Fan, Kettle, and Hairdryer. The total mixed load data of those appliance is 14 files. After that we performed data augmentation by giving noise to the signal, thus the total mixed load data becomes 238. Details about the number of each appliance can be seen in [Table 4].

[Table 4] Mixed Load Dataset from Private Dataset

Name of Target Load Appliance	Number of Data
Fan	56
Kettle	91
Hairdryer	91
Total Data	238

We also collected single load data of fan, kettle, and hairdryer. We also performed data augmentation for these signals. After data augmentation, total number of data is 120. Details number about each appliance is presented in [Table 5].

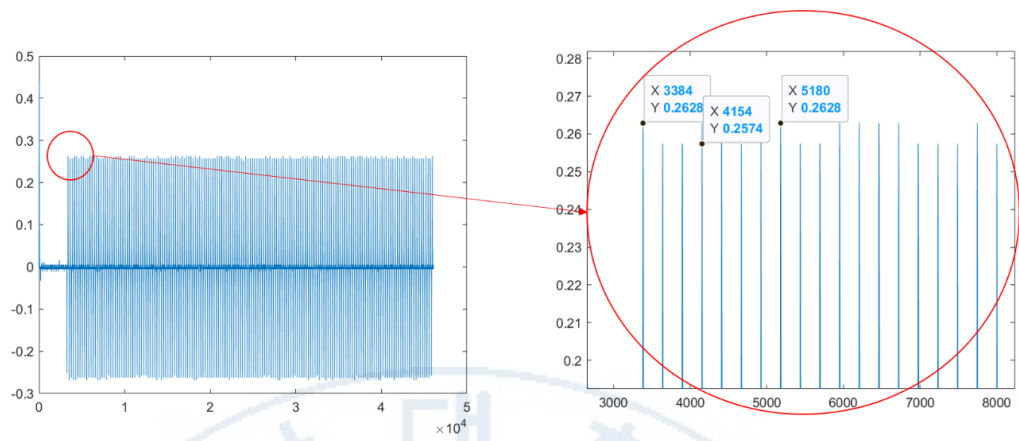
[Table 5] Single Load Dataset from Private Dataset

Name of appliance as target Load	Number of Data
Fan	40
Kettle	40
Hairdryer	40
Total Data	120

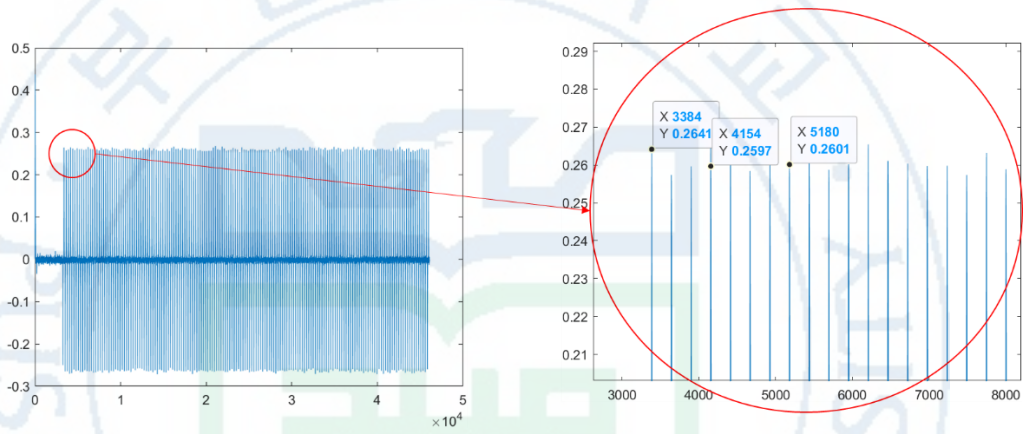
C. Data Preprocessing and Data Splitting Result

As mentioned in section 3, data preprocessing consists of several process either for single load dataset or mixed load dataset. Firstly, we marked the transient index. After that, only 3s before transient index and 3s after transient index was saved as waveform data. The 3s before transient index is the background load. The 3s after transient index is the mixed load. However, only 3s after the transient index was saved for single load data. This is because single load data does not have background load. After 6 s of mixed load data and 3 s of single load data was obtained, data augmentation and spectrogram transformation are performed.

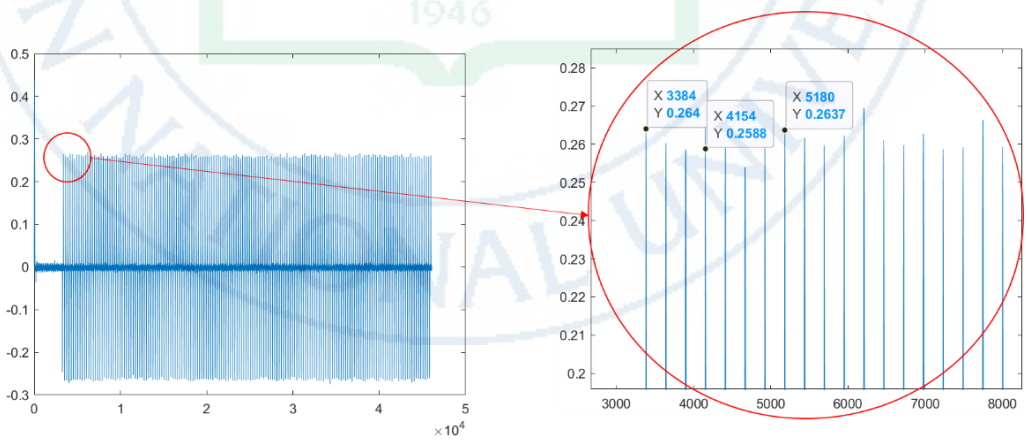
Data augmentation was performed for single load LIT-dataset, single load private data, and mixed load private dataset. Example of signal data before adding noise and after adding noise is presented in [Figure 26]. It can be seen that the value of every signal has changed and different each other. For example, in [Figure 26a], when X is 3384, Y is 0.2628; in [Figure 26b], when X is 3384, Y is 0.2641; in [Figure 26c], when X is 3384, Y is 0.264; in [Figure 26d], when X is 3384, Y is 0.2621; X denotes for time the value and Y denotes for current value. After augmentation, new data sample is generated with various values but same pattern.



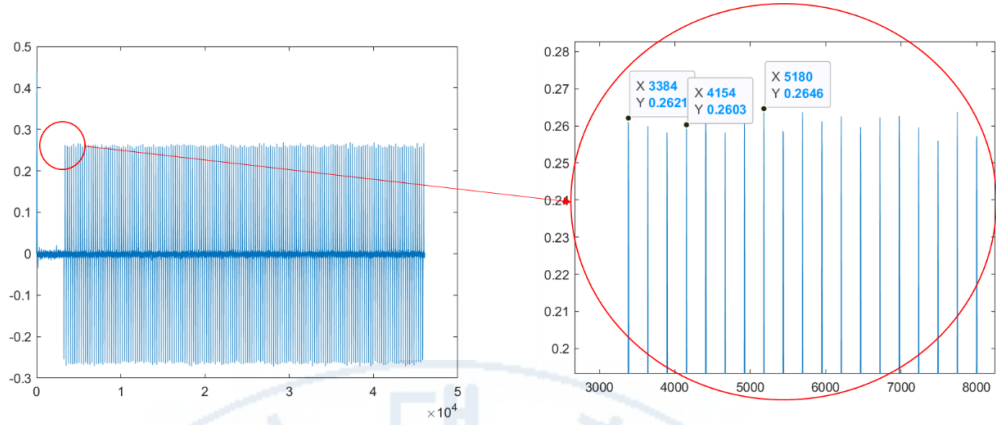
(a) Original Data of Single Load LED Lamp



(b) First Augmentation Result of Single Load LED Lamp



(c) Second Augmentation Result of Single Load LED Lamp



(d) Third Augmentation Result of Single Load LED Lamp

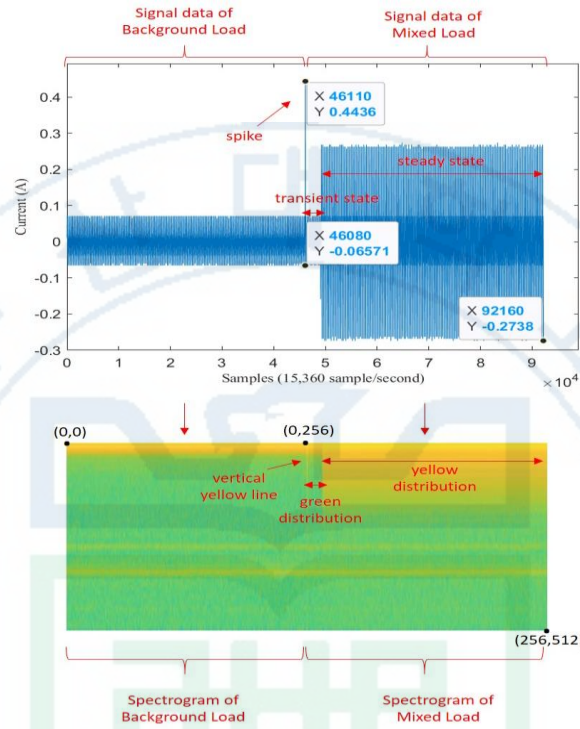
[Figure 26] Augmentation Results Comparison

A spectrogram image (in *.jpg format) was obtained following the process of STFT process. It came in the size of 512×256 for the mixed load dataset. In response, [Figure 27] exhibits a transformation of mixed load (contains LED Lamp) signal to spectrogram. It revealed transient index to take place in the middle of signal data. A contrast color was apparent in the center of spectrogram image, specifically at the 256th pixel. In such a circumstance, an LED lamp current signal turned on, exposing a short spike line at the starting time, indicating that the vertical line belongs to the mixed load. Following the event, a transient state was indicated by the current signal going down quickly. Subsequently, moved to a steady state, as it went up over higher ampere. The spectrogram image also explicated one yellow vertical line in the center of the image. It turned green for a moment and then back to yellow with more extensive distribution of pixels.

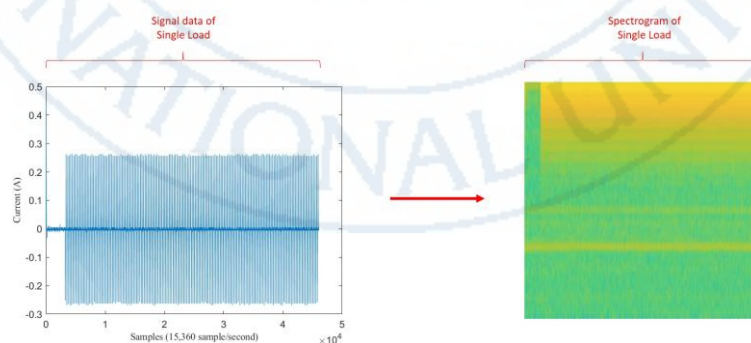
This result attests that spectrogram image represents the signal data adequately. On the left side, it signifies the spectrogram image of background load (S_b), while the right side signifies the spectrogram image of mixed load (S_m).

The single load dataset is also transformed into spectrogram image. But the size for a single load spectrogram image is only 256×256 because it has no background load. This 256×256 -sized spectrogram image represents 3 s or waveform load after turning-on event. This transformation of single load is illustrated in [Figure 28]. We

also chose an LED lamp for this illustration. Like the mixed load that contains LED lamp as target load, this LED lamp single load also displays one vertical spike at the beginning of the waveform. Corresponding to it, one vertical yellow line also apparent at the beginning of spectrogram image.

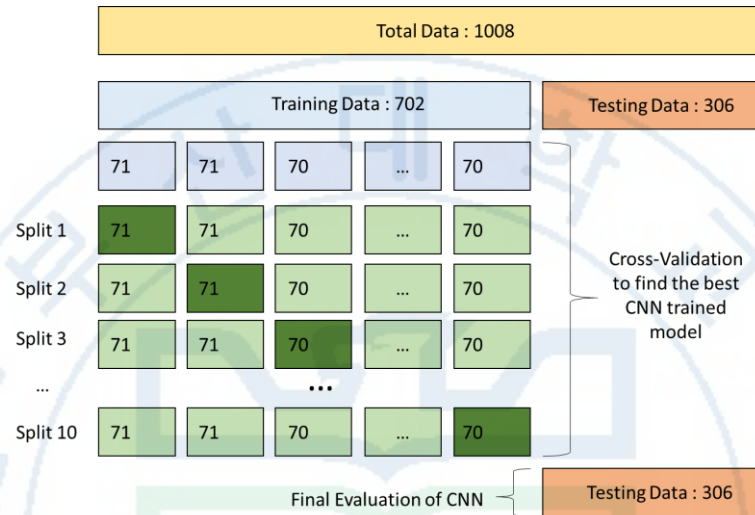


[Figure 27] Spectrogram Transformation of Mixed Load [10]

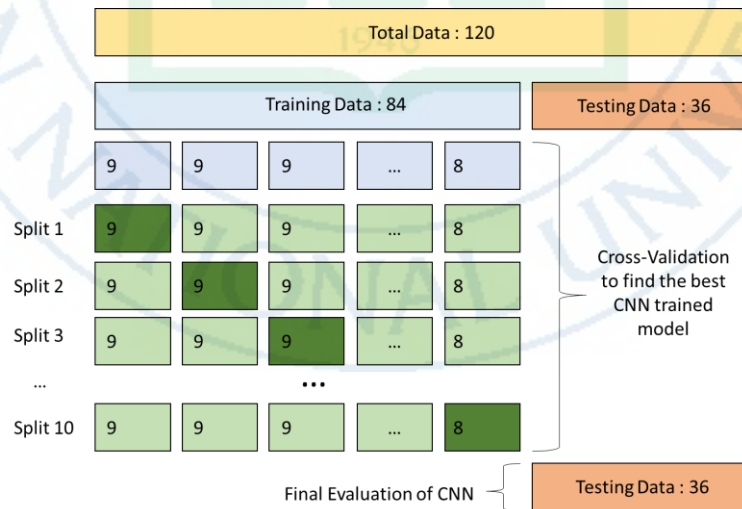


[Figure 28] Spectrogram Transformation of Single Load [10]

Data splitting was performed as explained in Section 3C. Single load data of LIT-Dataset was split using stratified k-fold. [Figure 29] illustrates the implementation of k-fold cross validation using LIT-dataset, while [Figure 30] illustrates the implementation of k-fold cross validation using private dataset.



[Figure 29] K-Fold Cross Validation Implemented in LIT-Dataset



[Figure 30] K-Fold Cross Validation Implemented in Private Dataset

Mixed load data of LIT-Dataset was split into 455 training data and 201 inference data (see [Table 6]). Mixed load data of Private Dataset was split into 165 training data and 73 inference data (see [Table 7]). The training dataset is used for GAN Model Generation, while the inference dataset is used for Inference Process.

[Table 6] Mixed Load Dataset Splitting - LIT Dataset

Splitting Part	Number of Data
Training Data	455
Inference data	201
Total Number	656

[Table 7] Mixed Load Dataset Splitting - Private Dataset

Splitting Part	Number of Data
Training Data	165
Inference data	73
Total Number	238

D. GAN Model Generation

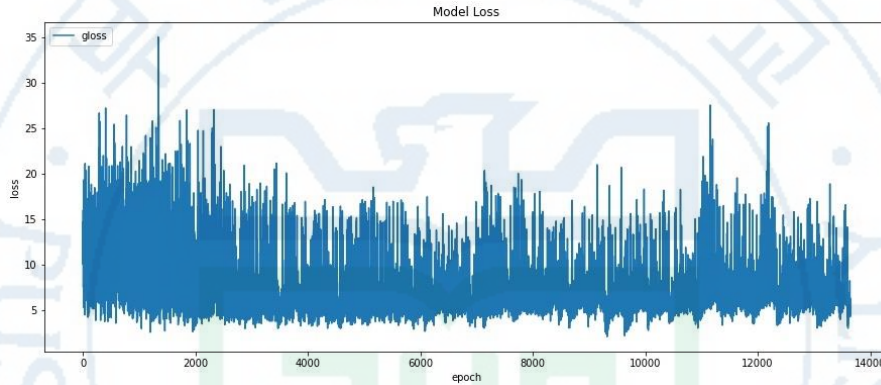
The GAN model was trained using 70% of mixed load dataset. Hyperparameter of GAN training can be seen in [Table 8].

[Table 8] GAN Model Hyperparameter

Hyperparameter	Value
Optimizer	Adam
Learning Rate (Alpha)	0.0002
Beta	0.5
Generator Loss Function	Cross-entropy and Mean Absolute Error (MAE)
Discriminator Loss Function	Binary Cross-Entropy

We implemented pix2pix GAN using Keras. The objective of Pix2pix GAN, which is the combination of cGAN objective and Loss L1, are implemented using Cross-entropy and Mean Absolute Error (MAE) in Keras.

In the code implementation, we denote the pix2pix GAN objective (G^*) as variable *gloss* to ease the visualization. [Figure 31] shows *gloss* curve representing the fluctuation of pix2pix GAN objective during training. At the beginning, gloss tends to be unstable and the fluctuation is uncertain. Starting from 4000th iteration, gloss range value remains stable in the range of 5 to 15. It means that at these iterations, generator is likely to produce the most plausible image.

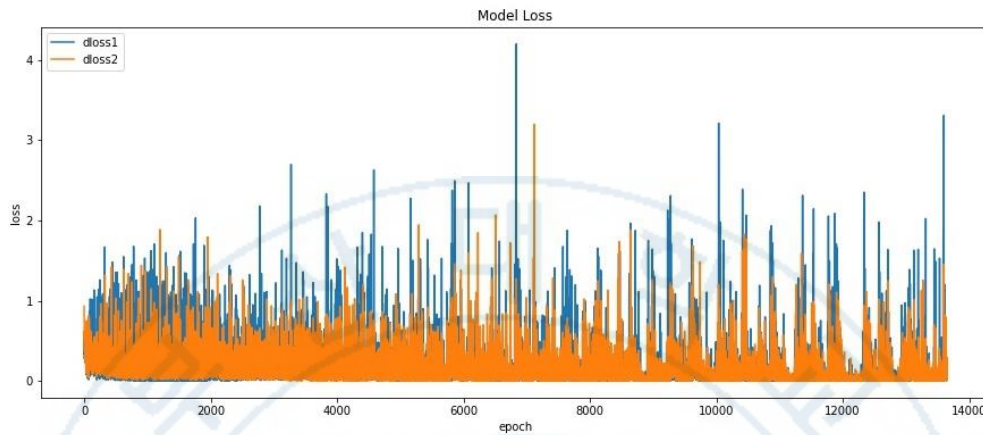


[Figure 31] Generator Loss Curve from GAN Training Result - LIT Dataset [10]

To know the performance of the discriminator, we plotted two types of discriminator loss. The first one is the loss that discriminating real images, called *dloss1*. The second one is the loss that determining fake images, called *dloss2*.

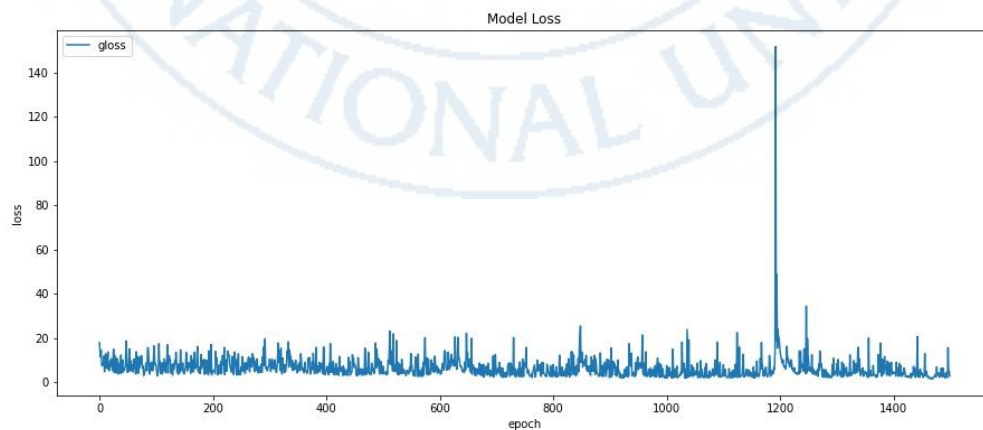
[Figure 32] presents the *dloss1* and *dloss2* plotting. We compared *dloss1* and *dloss2*. At the beginning, *dloss2* range values remain lower than *dloss1* range values. It means the discriminator is good in discriminating or identifying fake images (output of generator), but not good enough in determining real image. If discriminator is still good in identifying fake image, it means the generator is still poor in generating output image. However, *dloss1* and *dloss2* range values are nearly the same starting from 9000th iteration. It means the discriminator identifies real samples or fake samples equally. Therefore, the generator can be expected to have good performance in producing image after 9000th iteration. But, for specifically the iteration around 12,000

to 13,000 is likely to have the best generator model. It is because the *dloss1* and *dloss2* value are nearly the same at this range. That is the reason we took generator model at 13,650th iteration.

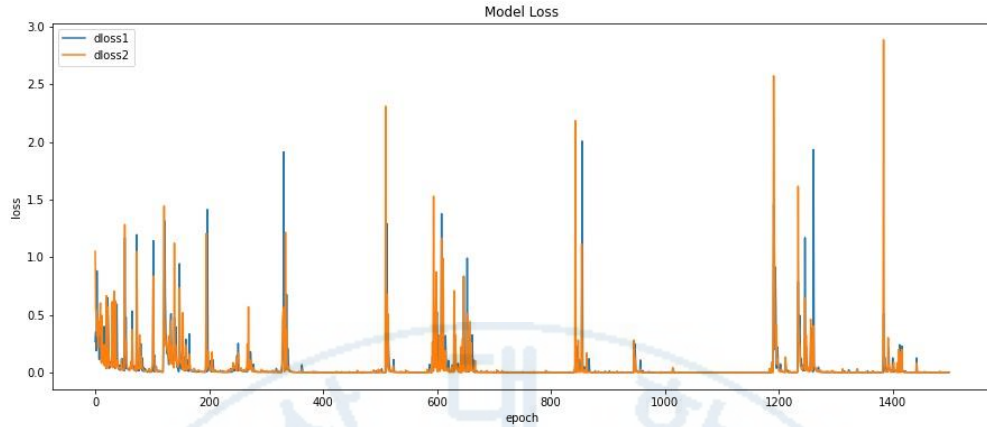


[Figure 32] Discriminator Loss from GAN Training Result - LIT dataset [10]

Using the same analysis as GAN training result in LIT-dataset, we also observed *gloss*, *dloss1*, and *dloss2* in the private dataset. In the private dataset, the chosen generator model is at the 1,500th iteration. At iteration 1,500th, the *dloss1* value and *dloss2* are 0.004 and 0.002, respectively. Those values are not much different. At the same time, *gloss* is also 2.88, which is considered low. [Figure 33] shows the *gloss* training curve while [Figure 34] shows the *dloss1* and *dloss2*.



[Figure 33] Generator Loss from GAN Training Result - Private Dataset



[Figure 34] Discriminator Loss GAN Training Result - Private Dataset

E. CNN Model Generation

We set hyperparameter as presented in [Table 9] to generate CNN Model. We utilized LIT-dataset and performed CNN training 10 times based on 10-fold cross validation as we mentioned in section 3. From this k-fold cross validation, we found the best data splitting between training and validation, to build the best model. The 10-fold result can be seen in

[Table 10]. Based on the result, there are 5 folds have highest accuracy which is 99.67%. Those are fold-2, fold-4, fold-5, fold-6, and fold-8. But we chose fold-2 to represent the best fold and used it for the rest experiment.

[Table 9] CNN Model Hyperparameter

Hyperparameter	Value
Optimizer	Adam
Learning Rate (Alpha)	0.0001
Epoch	180
Objective Function	Categorical Cross-entropy

[Table 10] CNN Cross Validation - LIT Dataset

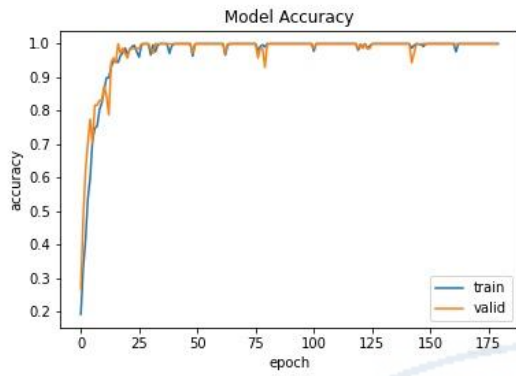
Fold	Accuracy (%)
1	99.02
2	99.67
3	99.02
4	99.67
5	99.67
6	99.67
7	99.35
8	99.67
9	99.35
10	99.35

After finding the best fold, we compared our model with several CNN architecture from ImageNet Challenge, such as VGG16, MobileNet, DenseNet121, InceptionV3, and Xception. These architectures were implemented using the same fold, environment, and hyperparameter setting.

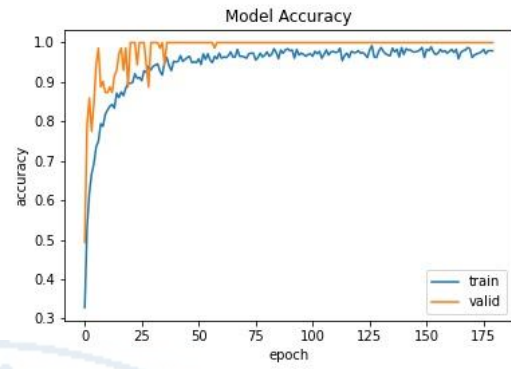
CNN Training Curve based on the accuracy is presented in [Figure 35]. It can be seen that only our proposed architecture and VGG16 are fitting well, while the other models are overfitting. It can also be confirmed by testing process (see [Table 11]). The testing was measured using “accuracy” metrics which can be calculated with Equation (15).

$$\left(\sum_{idx=1}^{Tot} X_{idx} / Tot \right) \times 100\% \quad (15)$$

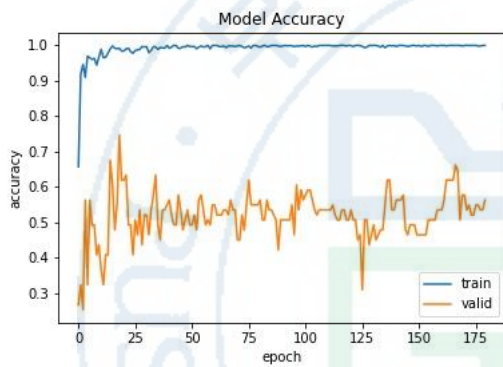
where $X_{idx} \in \{0,1\}$ and Tot is total data.



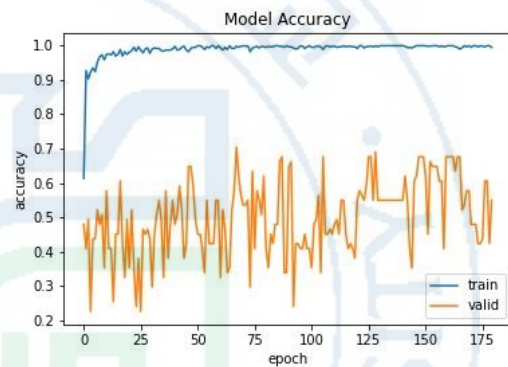
(a) Proposed Architecture



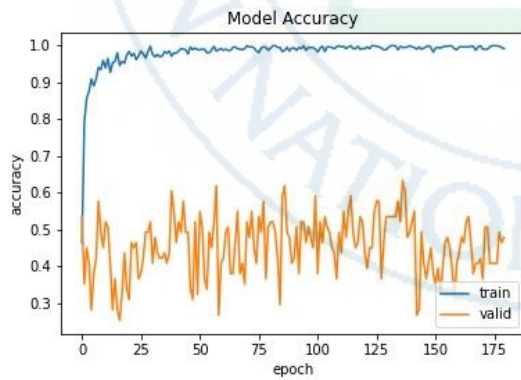
(b) VGG16



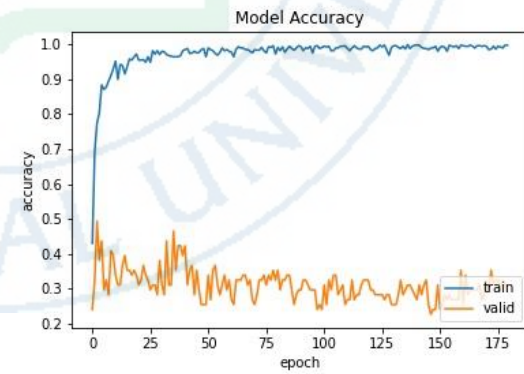
(c) MobileNet



(d) DenseNet121



(e) InceptionV3



(f) Xception

[Figure 35] CNN Training Result- LIT Dataset

[Table 11] CNN Testing Result - LIT Dataset

Model	Accuracy (%)					
	Proposed Architecture	VGG16	MobileNet	DenseNet121	InceptionV3	Xception
LED Lamp	100	100	5.88	0	100	2.94
LED Panel	100	100	82.35	100	97.06	73.53
Fume Extractor	100	100	0	0	0	2.94
Motorola Phone Charger	97.06	100	0	11.76	32.35	0
Fan	100	100	32.35	91.18	5.88	47.06
VIAO Laptop	100	100	100	73.53	29.41	8.82
Drill Speed 2	100	100	100	94.12	88.23	100
Microwave Oven ON	100	100	97.06	97.06	79.12	85.29
Hairdryer	100	100	50.00	32.35	14.70	0
Average	99.67	100	51.96	55.56	49.67	35.62

Based on [Table 11], It is only our proposed architecture and VGG16 having high accuracy, which are 99.67% and 100%, respectively. Although our proposed architecture accuracy is a little lower than VGG16, but it has so much less parameters. Total parameters of our proposed architecture are 989,321. While VGG16 is 138,423,208 [47]. Less number of parameters is better because it has less computation complexity. Thus, we can say that our proposed method is more beneficial. We also compare the testing time execution for our proposed architecture and VGG16. Our proposed architecture came out with 0.06 seconds, while VGG16 is 0.10 seconds. Our propose architecture is faster, which is more suitable for load identification. Based on those analysis, our proposed architecture was chosen as the base architecture of CNN load identification model for the rest of thesis. [Table 12] shows the detail comparison between our propose architecture and VGG16.

[Table 12] Comparison Between Proposed CNN Architecture and VGG16

Comparison Variable	Proposed CNN Architecture	VGG16
Average Accuracy	99.67%	100%
Parameter Number	989,321	138,423,208
Time Execution	0.06 s	0.10 s

The performance of CNN model affects the inference result of proposed two-step load identification method. Thus, our proposed architecture is chosen as the base architecture of CNN load identification model for the rest of paper.

After performing on LIT-Dataset, we also performed our proposed architecture on private dataset for confirmation. We also implemented stratified 10-fold cross validation for this private dataset. Among 10 folds, only fold-6 has highest accuracy which is 100%. The 10-fold cross validation of private dataset is presented in [Table 13]. The result of CNN testing on the private dataset is presented in [Table 14]. Our proposed architecture distinguishes 3 classes of appliances: Fan, Kettle, and Hairdryer. Same as LIT-Dataset, the models also performed well in private dataset. Our proposed CNN architecture reaches average accuracy 100%.

[Table 13] CNN Cross Validation - Private Dataset

Fold	Accuracy (%)
1	97.22
2	97.22
3	97.22
4	97.22
5	94.44
6	100
7	94.44
8	91.67
9	94.44
10	97.22

[Table 14] CNN Testing Result - Private Dataset

Appliance Name	Accuracy (%)
Fan	100
Kettle	100
Hairdryer	100
Average	100

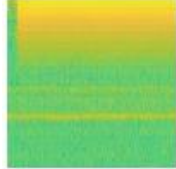
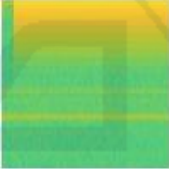
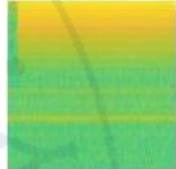
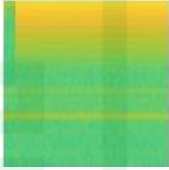
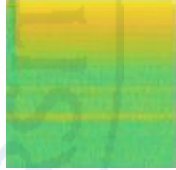
F. Inference

Inference process used 30% of mixed load dataset as we mentioned in Subsection 4C. This 30% mixed load data can be interpreted as new or unseen either by GAN training process or CNN training process. Inference process consists of Step-1 as denoising process and Step-2 as load identification process.

1. Step 1-Denoising

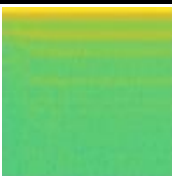
In the inference process, we used the trained GAN model to produce noise distribution. Using the noise distribution, we obtained a clean target load by subtracting mixed load with it. [Table 15] presents the overview of generated target load using the trained GAN. In the table, we present LED Lamp's target load generation for the example only. We analyze based on those results. The source images or input images are a combination of different types of background load. But all of them have the same target load, which is an LED lamp. In the first row of the table, the background load is microwave, and the target load is an LED lamp. In the second row, the background load is a soldering station, and the target load is an LED lamp. In the third row, the background load is a resistor, and the target load is an LED lamp. In the fourth row, the background load is oil heater and fan, and the target load is an LED lamp. Although all of them have various combinations of load, the generated target loads (last column) are all the same as the single loads. This analysis demonstrates that our trained Gan has succeeded in denoise various background load.

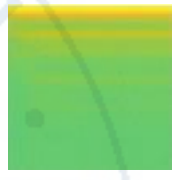
[Table 15] GAN Inference Result of LED Lamp - LIT Dataset

No	Sequence of Turning-on Appliance	Input Image	Single Load of LED Lamp	Generated Target Load of LED Lamp
1	• Microwave • LED Lamp			
2	• Soldering Station • LED Lamp			
3	• Resistor • LED Lamp			
4	• Oil heater • Fan • LED Lamp			

We also performed it in private dataset with target load hairdryer as the example. [Table 16] shows the result. Based on the table, the hairdryer load has various of background load, but the generated target loads are also nearly the same.

[Table 16] GAN Inference Result of Hairdryer - Private Dataset

No	Sequence of Turning-on Appliance	Input Image	Single Load of Hairdryer	Generated Target Load of Hairdryer
1	• Fan • Hairdryer			

No	Sequence of Turning-on Appliance	Input Image	Single Load of Hairdryer	Generated Target Load of Hairdryer
2	• SS • Kettle • Hairdryer			
3	• SS • LG • Hairdryer			
4	• SS • Hairdryer			

2. Step 2-Load Identification

In this step, the generated clean target load was identified using the selected CNN model. In this case, it is our proposed architecture. The result of load identification based on the LIT Dataset is presented in [Table 17], while the result of load identification based on the private dataset is presented in [Table 18]. Using the LIT dataset, our proposed architecture can reach 95.02% accuracy. Using the private dataset, our proposed architecture can reach 87.67% accuracy. The time execution for inference process in 0.38 seconds. Based on the result, it can be concluded that our proposed method is applicable to NILM load identification.

[Table 17] CNN Inference Result - LIT Dataset

Appliance Name	Accuracy (%)
LED Lamp	100
LED Panel	100
Fume Extractor	83.33

Appliance Name	Accuracy (%)
Motorola Phone Charger	100
Fan	50
VIAO Laptop	100
Drill Speed 2	100
Microwave Oven ON	93.33
Hairdryer	100
Average	95.02

[Table 18] CNN Inference Result - Private Dataset

Appliance Name	Accuracy (%)
Fan	94.12
Kettle	89.28
Hairdryer	82.14
Average	87.67

V. Conclusion

Energy conservation becomes one of the crucial issues in the world these days. This is because power consumption is always increasing, as the population's growth also always increases. Non-Intrusive Load Monitoring (NILM) is one of the systems that can contribute to energy conservation. Using NILM, an occupant can monitor individual appliance power consumption, which can lead to saving control.

One of the NILM objectives is load identification. Load identification is a classification task aiming to identify which appliance causes the turning-on event. In load identification, the acquisition load should be in a short window size after the turning-on event. This is because the fast control response is essential. Therefore, only a concise window of load is needed to be taken so the occupant will receive information in real time. Unfortunately, a short period signal easily submerges with background load, which results noises on the load that will be identified.

To solve that issue, we proposed two-step load identification method. The first step uses the Generative Adversarial Network (GAN) to denoise other appliances that are operating as background. The second step uses Convolutional Neural Network (CNN) to classify which appliance belongs to the turning-on load.

The GAN model was trained using mixed load dataset and noise distribution extracted from single load as the label. The experiments shown that the noise distribution has been learned successfully by the GAN model. Hence, regardless of the similarity of background load and mixed load, a clean target load can be extracted using that generated noise distribution by GAN. For CNN, we experimented using stratified k-fold cross validation and comparing our proposed architecture with other architectures like VGG16, MobileNet, DenseNet121, InceptionV3, and Xception. Based on the analysis of accuracy, parameter number, and time execution, we concluded that our proposed CNN is the best for load identification. The inference result using unseen data shown that our proposed method reach accuracy 95.02% for LIT-dataset

and 87.67% for private dataset. The time execution of this inference process is only 0.38 seconds.

Based on that result, we conclude that our proposed method is applicable to NILM load identification. In the future, this work can be implemented in a complete NILM system by combining event detection and load disaggregation module



References

- [1] S. Darby, "The Effectiveness of Feedback on Energy Consumption: A Review of the Literature on Metering, Billing and Direct Displays," Oxford, UK, 2006.
- [2] P. Meehan, C. McArdle, and S. Daniels, "An efficient, scalable time-frequency method for tracking energy usage of domestic appliances using a two-step classification algorithm," *Energies*, vol. 7, no. 11, pp. 7041–7066, 2014, doi: 10.3390/en7117041.
- [3] Q. Wu and F. Wang, "Concatenate convolutional neural networks for non-intrusive load monitoring across complex background," *Energies*, vol. 12, no. 8, 2019, doi: 10.3390/en12081572.
- [4] Y. H. Lin and M. S. Tsai, "Development of an improved time-frequency analysis-based nonintrusive load monitor for load demand identification," *IEEE Trans. Instrum. Meas.*, vol. 63, no. 6, pp. 1470–1483, 2014, doi: 10.1109/TIM.2013.2289700.
- [5] Y. Du, L. Du, B. Lu, R. Harley, and T. Habetler, "A review of identification and monitoring methods for electric loads in commercial and residential buildings," *2010 IEEE Energy Convers. Congr. Expo. ECCE 2010 - Proc.*, pp. 4527–4533, 2010, doi: 10.1109/ECCE.2010.5618423.
- [6] M. A. Ahajjam, D. B. Licea, M. Ghogho, and A. Kobbane, "IMPEC: An integrated system for monitoring and processing electricity consumption in buildings," *Sensors (Switzerland)*, vol. 20, no. 4, pp. 1–20, 2020, doi: 10.3390/s20041048.
- [7] A. A. Zaidi, F. Kupzog, T. Zia, and P. Palensky, "Load recognition for automated demand response in microgrids," *IECON Proc. (Industrial Electron. Conf.)*, pp. 2442–2447, 2010, doi: 10.1109/IECON.2010.5675022.
- [8] H. H. Chang, K. L. Lian, Y. C. Su, and W. J. Lee, "Power-spectrum-based wavelet transform for nonintrusive demand monitoring and load identification," *IEEE Trans. Ind. Appl.*, vol. 50, no. 3, pp. 2081–2089, 2014, doi: 10.1109/TIA.2013.2283318.

- [9] C. Gebbe, A. Bashir, and T. Neuhauser, "Supervised load identification of 18 fixed-speed motors based on their turn-on transient current," vol. XXX, no. Xxx, pp. 1-13.
- [10] A. Mukaroh, T. T. H. Le, and H. Kim, "Background load denoising across complex load based on generative adversarial network to enhance load identification," *Sensors (Switzerland)*, vol. 20, no. 19, pp. 1-23, 2020, doi: 10.3390/s20195674.
- [11] F. D. la Prieta *et al.*, "Trends in Cyber-Physical Multi-Agent Systems," 2017.
- [12] G. W. Hart, "Nonintrusive Appliance Load Monitoring," *Proc. IEEE*, vol. 80, no. 12, pp. 1870-1891, 1992, doi: 10.1109/5.192069.
- [13] L. S. M. Saif *et al.*, "Real-time appliances recognition for non-intrusive load monitoring using CNN," *J. Telecommun. Electron. Comput. Eng.*, vol. 10, no. 2-5, pp. 35-38, 2018.
- [14] A. Zoha, A. Gluhak, M. A. Imran, and S. Rajasegarar, "Non-intrusive Load Monitoring approaches for disaggregated energy sensing: A survey," *Sensors (Switzerland)*, vol. 12, no. 12, pp. 16838-16866, 2012, doi: 10.3390/s121216838.
- [15] M. Berges, E. Goldman, H. S. Matthews, L. Soibelman, and K. Anderson, "User-Centered Nonintrusive Electricity Load Monitoring for Residential Buildings," *J. Comput. Civ. Eng.*, vol. 25, no. 6, pp. 471-480, 2011, doi: 10.1061/(asce)cp.1943-5487.0000108.
- [16] L. K. Norford and S. B. Leeb, "Non-intrusive electrical load monitoring in commercial buildings based on steady-state and transient load-detection algorithms," *Energy Build.*, vol. 24, no. 1, pp. 51-64, 1996, doi: 10.1016/0378-7788(95)00958-2.
- [17] S. B. Leeb, S. R. Shaw, and J. L. Kirtley, "Transient Event Detection in Spectral Envelope Estimates for Nonintrusive Load Monitoring," *IEEE Trans. Power Deliv.*, vol. 10, no. 3, pp. 1200-1210, 1995, doi: 10.1109/61.400897.
- [18] T. T. H. Le, J. Kim, and H. Kim, "Classification performance using gated recurrent unit Recurrent Neural Network on energy disaggregation," *Proc. - Int. Conf. Mach. Learn. Cybern.*, vol. 1, no. 2010, pp. 105-110, 2016, doi:

10.1109/ICMLC.2016.7860885.

- [19] L. Yu, H. Li, X. Feng, and J. Duan, "Nonintrusive Appliance Load Monitoring for Smart Homes: Recent Advances and Future Issues," *IEEE Instrum. Meas. Mag.*, vol. 19, pp. 56–62, 2016.
- [20] A. Faustine, N. H. Mvungi, S. Kaijage, and K. Michael, "A survey on non-intrusive load monitoring methodologies and techniques for energy disaggregation problem," *arXiv*, 2017.
- [21] O. Parson, S. Ghosh, M. Weal, and A. Rogers, "Non-intrusive load monitoring using prior models of general appliance types," *Proc. Natl. Conf. Artif. Intell.*, vol. 1, pp. 356–362, 2012.
- [22] J. Liao, G. Elafoudi, L. Stankovic, and V. Stankovic, "Non-intrusive appliance load monitoring using low-resolution smart meter data," *2014 IEEE Int. Conf. Smart Grid Commun. SmartGridComm 2014*, no. November, pp. 535–540, 2015, doi: 10.1109/SmartGridComm.2014.7007702.
- [23] M. Zeifman and K. Roth, "Nonintrusive appliance load monitoring: Review and outlook," *IEEE Trans. Consum. Electron.*, vol. 57, no. 1, pp. 76–84, 2011, doi: 10.1109/TCE.2011.5735484.
- [24] S. Makonin, F. Popowich, L. Bartram, B. Gill, and I. V. Bajić, "AMPds: A public dataset for load disaggregation and eco-feedback research," *2013 IEEE Electr. Power Energy Conf. EPEC 2013*, no. EPEC, 2013, doi: 10.1109/EPEC.2013.6802949.
- [25] K. S. Barsim, L. Mauch, and B. Yang, "Neural network ensembles to real-time identification of plug-level appliance measurements," *arXiv*, 2018.
- [26] J. Z. Kolter and T. Jaakkola, "Approximate inference in additive factorial HMMs with application to energy disaggregation," *J. Mach. Learn. Res.*, vol. 22, pp. 1472–1482, 2012.
- [27] B. Zhao, L. Stankovic, and V. Stankovic, "Blind non-intrusive appliance load monitoring using graph-based signal processing," *2015 IEEE Glob. Conf. Signal Inf. Process. Glob. 2015*, pp. 68–72, 2016, doi: 10.1109/GlobalSIP.2015.7418158.

- [28] K. He, L. Stankovic, J. Liao, and V. Stankovic, "Non-Intrusive Load Disaggregation Using Graph Signal Processing," *IEEE Trans. Smart Grid*, vol. 9, no. 3, pp. 1739–1747, 2018, doi: 10.1109/TSG.2016.2598872.
- [29] I. J. Goodfellow *et al.*, "Generative adversarial nets," *Adv. Neural Inf. Process. Syst.*, vol. 3, no. January, pp. 2672–2680, 2014.
- [30] M. Alhamid, "Generative Adversarial Networks GANs: A Beginner's Guide," 2019. <https://towardsdatascience.com/generative-adversarial-networks-gans-a-beginners-guide-f37c9f3b7817> (accessed Nov. 01, 2020).
- [31] M. Mirza and S. Osindero, "Conditional Generative Adversarial Nets," pp. 1–7, 2014, [Online]. Available: <http://arxiv.org/abs/1411.1784>.
- [32] P. Isola, J. Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," *Proc. - 30th IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2017*, vol. 2017-Janua, pp. 5967–5976, 2017, doi: 10.1109/CVPR.2017.632.
- [33] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, "Context Encoders: Feature Learning by Inpainting," *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2016-Decem, pp. 2536–2544, 2016, doi: 10.1109/CVPR.2016.278.
- [34] D. Cornelisse, "An intuitive guide to Convolutional Neural Networks," 2018. <https://www.freecodecamp.org/news/an-intuitive-guide-to-convolutional-neural-networks-260c2de0a050/> (accessed Nov. 09, 2020).
- [35] A. Satt, S. Rozenberg, and R. Hoory, "Efficient emotion recognition from speech using deep learning on spectrograms," *Proc. Annu. Conf. Int. Speech Commun. Assoc. INTERSPEECH*, vol. 2017-Augus, pp. 1089–1093, 2017, doi: 10.21437/Interspeech.2017-200.
- [36] D. C. Costa, G. A. M. Lopes, C. A. B. Mello, and H. O. Viana, "Speech and phoneme segmentation under noisy environment through spectrogram image analysis," *Conf. Proc. - IEEE Int. Conf. Syst. Man Cybern.*, pp. 1017–1022, 2012, doi: 10.1109/ICSMC.2012.6377862.

- [37] L. Yuan and J. Cao, "Patients' EEG Data Analysis via Spectrogram Image," *Int. Conf. Intell. Decis. Technol.*, pp. 13-21, 2018, doi: 10.1007/978-3-319-59421-7.
- [38] F. Zhu, G. Chen, and P. A. Heng, "From noise modeling to blind image denoising," *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2016-Decem, pp. 420-429, 2016, doi: 10.1109/CVPR.2016.52.
- [39] A. Fawzi, H. Samulowitz, D. Turaga, and P. Frossard, "Adaptive data augmentation for image classification," *Proc. - Int. Conf. Image Process. ICIP*, vol. 2016-Augus, pp. 3688-3692, 2016, doi: 10.1109/ICIP.2016.7533048.
- [40] F. Wang, S. H. Zhong, J. Peng, J. Jiang, and Y. Liu, "Data augmentation for eeg-based emotion recognition with deep convolutional neural networks," *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 10705 LNCS, pp. 82-93, 2018, doi: 10.1007/978-3-319-73600-6_8.
- [41] Q. Chaudhari, "Additive White Gaussian Noise (AWGN)," 2018. <https://wirelesspi.com/additive-white-gaussian-noise-awgn/> (accessed Nov. 10, 2020).
- [42] J. P. Linnartz, "Wireless Communication Reference Website." <http://www.wirelesscommunication.nl/reference/chaptr05/digimod/awgn.htm> (accessed Nov. 12, 2020).
- [43] H. Pishro-Nik, *Introduction to probability, statistics, and random processes*. Kappa Research LLC, 2014.
- [44] N. Kehtarnavaz, *Digital Signal Processing System Design*, 2nd ed. Academic Press, 2008.
- [45] P. Boersma and D. Weenink, "Viewing a spectrogram," 2011. https://www.fon.hum.uva.nl/praat/manual/Intro_3_1__Viewing_a_spectrogram.html (accessed Nov. 16, 2020).
- [46] D. P. B. Renaux *et al.*, "Designing a novel dataset for non-intrusive load monitoring," *Brazilian Symp. Comput. Syst. Eng. SBESC*, vol. 2018-Novem, pp. 243-249, 2018, doi: 10.1109/SBESC.2018.00045.

- [47] A. Anwar, "Difference between AlexNet, VGGNet, ResNet, and Inception," 2019. <https://towardsdatascience.com/the-w3h-of-alexnet-vggnet-resnet-and-inception-7baaaecccc96#:~:text=VGG16 has a total of,with a stride of two.> (accessed Nov. 17, 2020).



NILM 시스템을 위한 GAN 및 CNN 기반 2단계 부하 식별 방법

아피파툴 무카로¹

부산대학교 대학원 정보융합공학과

요약

세계의 에너지 소비는 증가하고 있다. 결과적으로, 에너지 부족은 특히 전력 전기 수요가 가용 에너지보다 높을 때 발생할 수 있다. 그런 식으로 에너지 절약은 걱정거리 중의 하나가 된다. NILM(Non-Intrusive Load Monitoring)은 그에 상응하는 에너지 절약에 기여하는 시스템이다. 각 전기기기의 전력사용량을 NILM을 이용해 모니터링할 수 있기 때문이다. 모니터링을 통해 사용자의 전력 소비량에 대한 인지도가 높아지면 전력 소비량이 줄어든다. 불행하게도, NILM에는 아직도 몇 가지 이슈가 있다. 그 중 하나는 복잡한 배경 부하에 걸친 부하 식별이다. 다른 부하에 걸쳐 동시에 작동하는 켜기 장치를 식별하는 것은 과제가 된다. 이는 다른 부하가 부하 식별 모델 성능을 저하시킬 수 있는 소음일 수 있기 때문이다. 따라서 본 논문에서는 2단계 하중 식별 방법을 제안하였다. 첫 번째 단계는 GAN(Generative Adversarial Network)을 사용하여 백그라운드에서 작동하는 다른 어플라이언스의 청색을 제거하는 것이다. 두 번째 단계는 CNN(Convolutional Neural Network)을 사용하여 켜기 부하에 속하는 어플라이언스를 분류하는 것이다. 이 접근방식은 Lab에서 수집된 LIT의 공개 데이터 세트와 개인 데이터 세트를 사용하여 실험했다. 실험 결과는 제안된 방법의 놀라운 성능을 입증한다.

¹ 본 논문작성자는 한국정부초청장학금(Global Korea Scholarship)을 지원받은 장학생임